

# Graphical Representation in Python

Technical Notes

Rodrigo Kang

## Table of contents

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Graphical Representation</b>                         | <b>4</b>  |
| <b>2</b> | <b>From Data to Visual Representation</b>               | <b>6</b>  |
| 2.1      | Common Visual Encodings . . . . .                       | 8         |
| <b>3</b> | <b>Matplotlib</b>                                       | <b>9</b>  |
| <b>4</b> | <b>Figures and Axes</b>                                 | <b>10</b> |
| 4.1      | Adding Labels and Titles . . . . .                      | 12        |
| 4.2      | Legends . . . . .                                       | 14        |
| <b>5</b> | <b>Scatter Plots</b>                                    | <b>15</b> |
| 5.1      | Association Is Not Causation . . . . .                  | 17        |
| 5.2      | Representing Groups . . . . .                           | 18        |
| 5.3      | Marker Size . . . . .                                   | 20        |
| 5.4      | Colour Maps . . . . .                                   | 21        |
| 5.5      | Overplotting . . . . .                                  | 22        |
| <b>6</b> | <b>Line Plots</b>                                       | <b>24</b> |
| 6.1      | Lines Imply Continuity or Ordered Progression . . . . . | 25        |
| 6.2      | Multiple Series . . . . .                               | 26        |
| 6.3      | Line Style and Markers . . . . .                        | 27        |
| 6.4      | Mathematical Functions . . . . .                        | 28        |
| 6.5      | Scatter Plot or Line Plot? . . . . .                    | 30        |
| <b>7</b> | <b>Bar Plots</b>                                        | <b>31</b> |
| 7.1      | Bar Plots and Aggregated Data . . . . .                 | 32        |
| 7.2      | Horizontal Bar Plots . . . . .                          | 34        |
| 7.3      | Ordering Categories . . . . .                           | 35        |
| 7.4      | Baselines and Bar Length . . . . .                      | 37        |
| <b>8</b> | <b>Histograms</b>                                       | <b>37</b> |
| 8.1      | Bins . . . . .                                          | 38        |

|           |                                                        |           |
|-----------|--------------------------------------------------------|-----------|
| 8.2       | What a Histogram Can Reveal . . . . .                  | 41        |
| 8.3       | Frequency and Density . . . . .                        | 43        |
| <b>9</b>  | <b>Box Plots</b>                                       | <b>44</b> |
| 9.1       | Whiskers and Potential Outliers . . . . .              | 46        |
| 9.2       | Comparing Groups . . . . .                             | 47        |
| <b>10</b> | <b>Bar Plot, Histogram, or Box Plot?</b>               | <b>48</b> |
| <b>11</b> | <b>Multiple Plots and Subplots</b>                     | <b>49</b> |
| 11.1      | Two-Dimensional Axes Arrays . . . . .                  | 51        |
| 11.2      | Shared Axes . . . . .                                  | 53        |
| 11.3      | Subplots Versus Overlaid Series . . . . .              | 54        |
| <b>12</b> | <b>Visual Encoding and Scales</b>                      | <b>54</b> |
| 12.1      | Position . . . . .                                     | 55        |
| 12.2      | Length . . . . .                                       | 55        |
| 12.3      | Colour . . . . .                                       | 55        |
| 12.4      | Size . . . . .                                         | 57        |
| 12.5      | Transparency . . . . .                                 | 58        |
| <b>13</b> | <b>Axis Scales</b>                                     | <b>59</b> |
| 13.1      | Logarithmic Scales . . . . .                           | 60        |
| <b>14</b> | <b>Time-Series Visualization</b>                       | <b>62</b> |
| 14.1      | Time as an Ordered Dimension . . . . .                 | 64        |
| 14.2      | Trend . . . . .                                        | 65        |
| 14.3      | Seasonality . . . . .                                  | 66        |
| 14.4      | Multiple Time Series . . . . .                         | 67        |
| 14.5      | Time-Series Plots and Causality . . . . .              | 69        |
| <b>15</b> | <b>Seaborn and Statistical Visualization</b>           | <b>70</b> |
| 15.1      | Semantic Mappings . . . . .                            | 72        |
| <b>16</b> | <b>Statistical Relationships with Seaborn</b>          | <b>73</b> |
| <b>17</b> | <b>Visualizing Distributions with Seaborn</b>          | <b>75</b> |
| 17.1      | Box Plots by Group . . . . .                           | 77        |
| <b>18</b> | <b>Faceting</b>                                        | <b>79</b> |
| <b>19</b> | <b>Choosing an Appropriate Visualization</b>           | <b>80</b> |
| 19.1      | One Numerical Variable . . . . .                       | 81        |
| 19.2      | One Categorical and One Numerical Variable . . . . .   | 81        |
| 19.3      | Two Numerical Variables . . . . .                      | 81        |
| 19.4      | Numerical Variable Over an Ordered Dimension . . . . . | 82        |
| 19.5      | Several Numerical Variables . . . . .                  | 82        |
| <b>20</b> | <b>Pie Charts</b>                                      | <b>82</b> |

|                                                                    |            |
|--------------------------------------------------------------------|------------|
| <b>21 Common Visualization Problems</b>                            | <b>85</b>  |
| 21.1 Truncated Axes . . . . .                                      | 86         |
| 21.2 Excessive Visual Complexity . . . . .                         | 86         |
| 21.3 Overplotting . . . . .                                        | 87         |
| 21.4 Inappropriate Aggregation . . . . .                           | 87         |
| 21.5 Dual Axes . . . . .                                           | 87         |
| 21.6 Uncertainty . . . . .                                         | 88         |
| 21.7 Graphical Association and Causal Interpretation . . . . .     | 88         |
| <b>22 Network Visualization</b>                                    | <b>89</b>  |
| 22.1 Creating a Graph . . . . .                                    | 89         |
| 22.2 Neighbours and Degree . . . . .                               | 90         |
| 22.3 Weighted Edges . . . . .                                      | 91         |
| 22.4 Drawing a Graph . . . . .                                     | 92         |
| <b>23 Visualization in Data Science and Machine Learning</b>       | <b>94</b>  |
| 23.1 Data Understanding . . . . .                                  | 94         |
| 23.2 Model Diagnostics . . . . .                                   | 94         |
| 23.3 Model Comparison . . . . .                                    | 96         |
| 23.4 Learning Curves . . . . .                                     | 98         |
| 23.5 Visualization as Part of Analytical Reasoning . . . . .       | 100        |
| <b>24 Key Takeaways</b>                                            | <b>101</b> |
| <b>25 Problems</b>                                                 | <b>102</b> |
| 25.1 Problem 1 — Basic Line Plot . . . . .                         | 102        |
| 25.2 Problem 2 — Visualizing a Mathematical Function . . . . .     | 103        |
| 25.3 Problem 3 — Comparing Mathematical Functions . . . . .        | 103        |
| 25.4 Problem 4 — Figure Configuration . . . . .                    | 104        |
| 25.5 Problem 5 — Scatter Plot . . . . .                            | 104        |
| 25.6 Problem 6 — Adding a Group Encoding . . . . .                 | 105        |
| 25.7 Problem 7 — Continuous Colour Encoding . . . . .              | 105        |
| 25.8 Problem 8 — Marker Size as an Encoding . . . . .              | 106        |
| 25.9 Problem 9 — Overplotting . . . . .                            | 106        |
| 25.10 Problem 10 — Bar Plot . . . . .                              | 107        |
| 25.11 Problem 11 — Truncated Bar Chart . . . . .                   | 107        |
| 25.12 Problem 12 — Histogram and Bin Selection . . . . .           | 108        |
| 25.13 Problem 13 — Distribution Shape . . . . .                    | 108        |
| 25.14 Problem 14 — Frequency Versus Density . . . . .              | 109        |
| 25.15 Problem 15 — Box Plot . . . . .                              | 109        |
| 25.16 Problem 16 — Comparing Distributions Across Groups . . . . . | 110        |
| 25.17 Problem 17 — Bar Plot or Box Plot? . . . . .                 | 111        |
| 25.18 Problem 18 — Multiple Subplots . . . . .                     | 111        |
| 25.19 Problem 19 — Linear and Logarithmic Scales . . . . .         | 112        |
| 25.20 Problem 20 — Time-Series Plot . . . . .                      | 112        |
| 25.21 Problem 21 — Trend and Seasonality . . . . .                 | 113        |
| 25.22 Problem 22 — Multiple Time Series . . . . .                  | 113        |

|       |                                                                     |     |
|-------|---------------------------------------------------------------------|-----|
| 25.23 | Problem 23 — Seaborn Scatter Plot . . . . .                         | 114 |
| 25.24 | Problem 24 — Regression Visualization . . . . .                     | 114 |
| 25.25 | Problem 25 — Histogram and Density Estimate . . . . .               | 114 |
| 25.26 | Problem 26 — Faceting . . . . .                                     | 115 |
| 25.27 | Problem 27 — Choosing the Appropriate Plot . . . . .                | 115 |
| 25.28 | Problem 28 — Pie Chart Versus Bar Chart . . . . .                   | 116 |
| 25.29 | Problem 29 — Misleading Visualization . . . . .                     | 116 |
| 25.30 | Problem 30 — Aggregation Can Hide Structure . . . . .               | 117 |
| 25.31 | Problem 31 — Uncertainty . . . . .                                  | 117 |
| 25.32 | Problem 32 — Three-Dimensional Parametric Plot . . . . .            | 118 |
| 25.33 | Problem 33 — Three-Dimensional Scatter Plot . . . . .               | 118 |
| 25.34 | Problem 34 — Creating a Meshgrid . . . . .                          | 119 |
| 25.35 | Problem 35 — Surface Plot . . . . .                                 | 119 |
| 25.36 | Problem 36 — Surface and Wireframe Comparison . . . . .             | 120 |
| 25.37 | Problem 37 — Network Construction . . . . .                         | 120 |
| 25.38 | Problem 38 — Network Layout . . . . .                               | 121 |
| 25.39 | Problem 39 — Weighted Network . . . . .                             | 121 |
| 25.40 | Problem 40 — Residual Plot . . . . .                                | 121 |
| 25.41 | Problem 41 — Detecting Nonlinearity from Residuals . . . . .        | 122 |
| 25.42 | Problem 42 — Model Comparison . . . . .                             | 123 |
| 25.43 | Problem 43 — Learning Curves . . . . .                              | 123 |
| 25.44 | Problem 44 — Visualizing Class Imbalance . . . . .                  | 124 |
| 25.45 | Problem 45 — Visual Encoding Critique . . . . .                     | 125 |
| 25.46 | Problem 46 — Dual-Axis Critique . . . . .                           | 125 |
| 25.47 | Problem 47 — Correlation and Causality . . . . .                    | 125 |
| 25.48 | Problem 48 — Design an Exploratory Visualization Strategy . . . . . | 126 |
| 25.49 | Problem 49 — Diagnose a Poor Visualization . . . . .                | 126 |
| 25.50 | Problem 50 — Integrated Visualization Reasoning . . . . .           | 127 |

## 1 Graphical Representation

Data analysis is not only a numerical process. Many structures that are difficult to identify from tables or summary statistics become immediately apparent when data are represented graphically.

A visualization can reveal:

- relationships between variables;
- differences between groups;
- distributions and skewness;
- outliers and unusual observations;
- temporal patterns;
- nonlinear structure;
- changes in variability;

- potential data-quality problems.

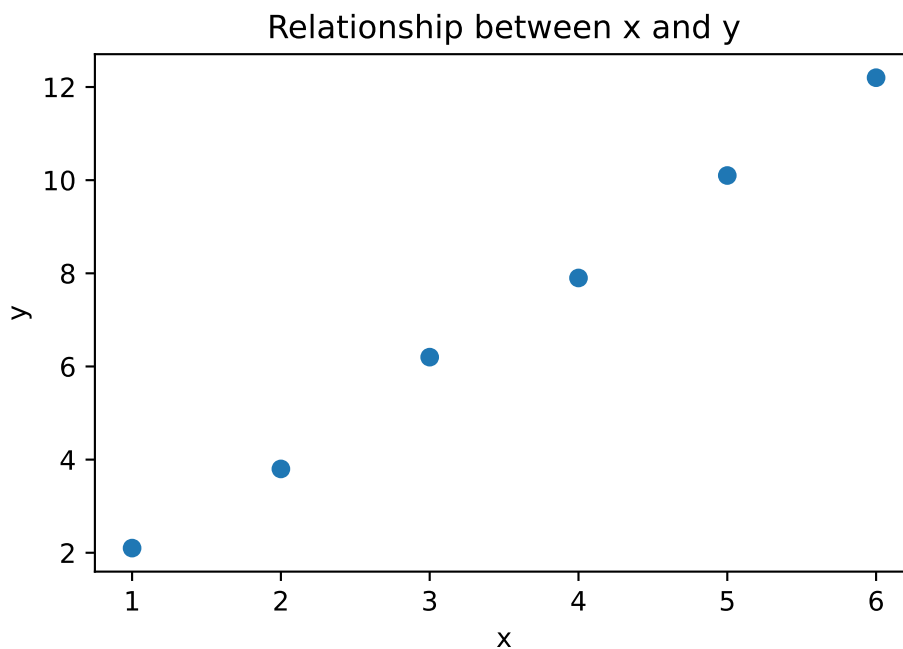
For this reason, graphical representation is an important part of **exploratory data analysis**, statistical modelling, and machine learning.

Consider two numerical variables:

```
1 import numpy as np
2
3 x = np.array([1, 2, 3, 4, 5, 6])
4 y = np.array([2.1, 3.8, 6.2, 7.9, 10.1, 12.2])
```

The numerical values contain all the information, but their relationship becomes easier to perceive when plotted.

```
1 import matplotlib.pyplot as plt
2
3 fig, ax = plt.subplots()
4
5 ax.scatter(x, y)
6
7 ax.set(
8     title="Relationship between x and y",
9     xlabel="x",
10    ylabel="y"
11 )
12
13 plt.show()
```



The plot makes the positive relationship between the variables visually apparent.

A graphical representation should therefore not be considered merely a decorative output. It is another representation of the data, designed to make particular structures easier to perceive.

However, every visualization is selective. Decisions about axes, scales, colours, grouping, aggregation, and graphical form determine which aspects of the data become prominent.

A useful visualization therefore begins with a question:

**What property of the data should the graphic make easier to understand?**

The appropriate graphical representation depends on that question.

## 2 From Data to Visual Representation

A statistical graphic maps properties of data to **visual properties**.

Suppose a dataset contains observations described by two numerical variables:

$$(x_i, y_i), \quad i = 1, \dots, n.$$

A scatter plot maps:

$$x_i \longrightarrow \text{horizontal position}$$

and:

$y_i \longrightarrow$  vertical position.

Additional variables could be represented using other visual properties, such as:

- colour;
- marker shape;
- marker size;
- transparency.

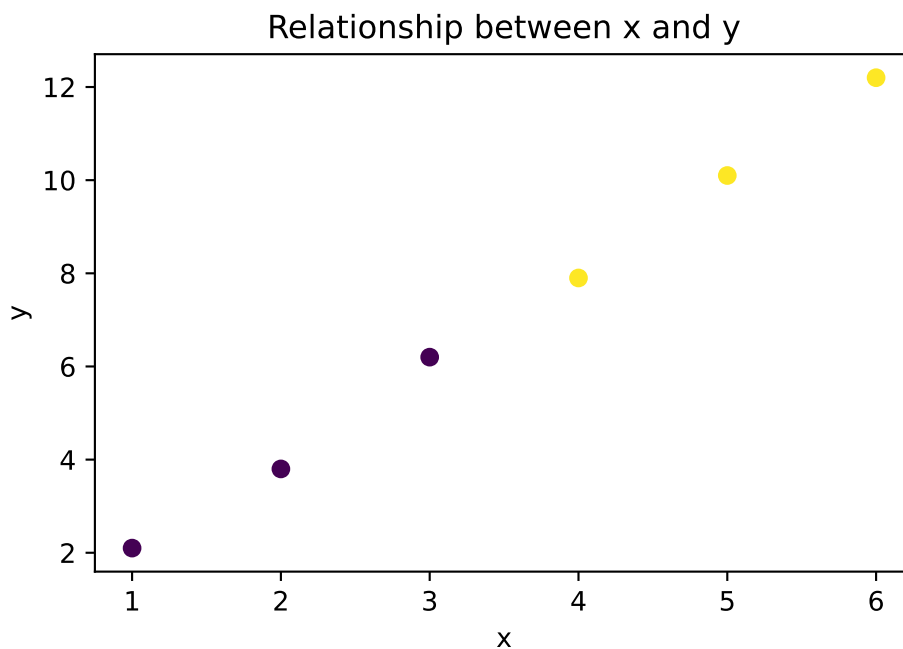
These properties are often called **visual encodings**.

For example, suppose each observation also belongs to a group:

```
1 x = np.array([1, 2, 3, 4, 5, 6])
2 y = np.array([2.1, 3.8, 6.2, 7.9, 10.1, 12.2])
3 group = np.array([0, 0, 0, 1, 1, 1])
```

The group could be encoded using colour:

```
1 fig, ax = plt.subplots()
2
3 scatter = ax.scatter(
4     x,
5     y,
6     c=group,
7     cmap="viridis"
8 )
9
10 ax.set(
11     title="Relationship between x and y",
12     xlabel="x",
13     ylabel="y"
14 )
15
16 plt.show()
```



The resulting graphic contains more information than the original two-dimensional scatter plot because colour now represents an additional variable.

The important point is not the particular colour map. It is the mapping:

data variable  $\rightarrow$  visual property.

Understanding this mapping helps distinguish meaningful graphical design from arbitrary styling.

## 2.1 Common Visual Encodings

Different graphical properties are appropriate for different types of information.

**Position** is particularly effective for representing numerical values. This is why scatter plots, line plots, and many statistical graphics rely heavily on horizontal and vertical position.

**Length** is commonly used in bar charts.

**Colour** can distinguish groups or represent numerical magnitude.

**Marker shape** can distinguish a relatively small number of categories.

**Marker size** can represent another numerical quantity, although comparisons based on area are generally less precise than comparisons based on position.

**Transparency** can help reveal observation density when many points overlap.



The choice of encoding should therefore follow the structure of the data and the analytical purpose of the visualization.

### 3 Matplotlib

Matplotlib is one of the foundational visualization libraries in the Python scientific ecosystem.

The conventional import is:

```
1 import matplotlib.pyplot as plt
```

matplotlib.pyplot provides functions for constructing and displaying plots.

There are two common ways of interacting with Matplotlib:

1. the state-based pyplot interface;
2. the object-oriented interface based on Figure and Axes objects.

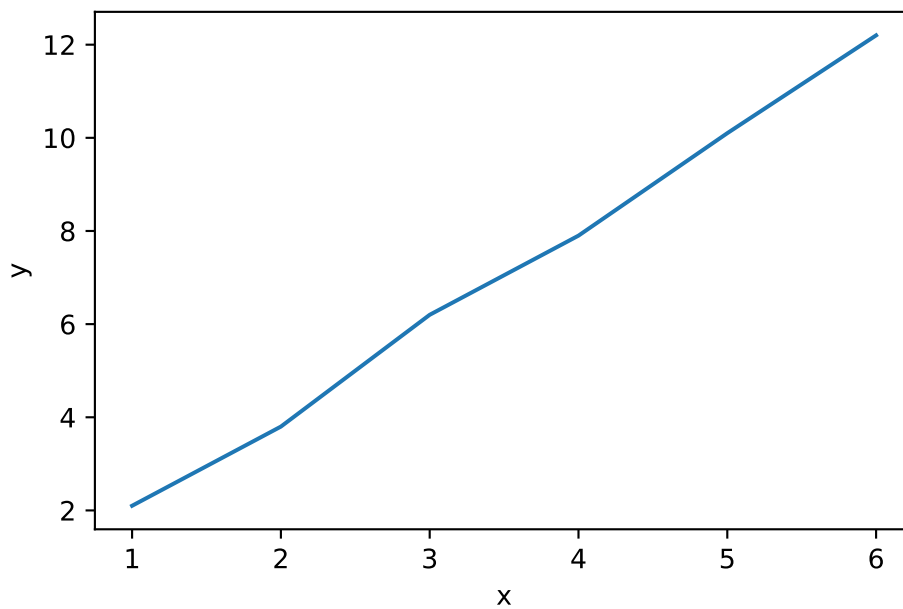
For simple interactive work, code such as:

```
1 plt.plot(x, y)
2 plt.xlabel("x")
3 plt.ylabel("y")
4 plt.show()
```

is valid and common.

For structured analytical code, however, the object-oriented interface is often clearer:

```
1 fig, ax = plt.subplots()
2
3 ax.plot(x, y)
4
5 ax.set(
6     xlabel="x",
7     ylabel="y"
8 )
9
10 plt.show()
```



This style makes explicit which graphical object is being modified and scales naturally to figures containing multiple plots.

These notes will therefore use the object-oriented interface as the default.

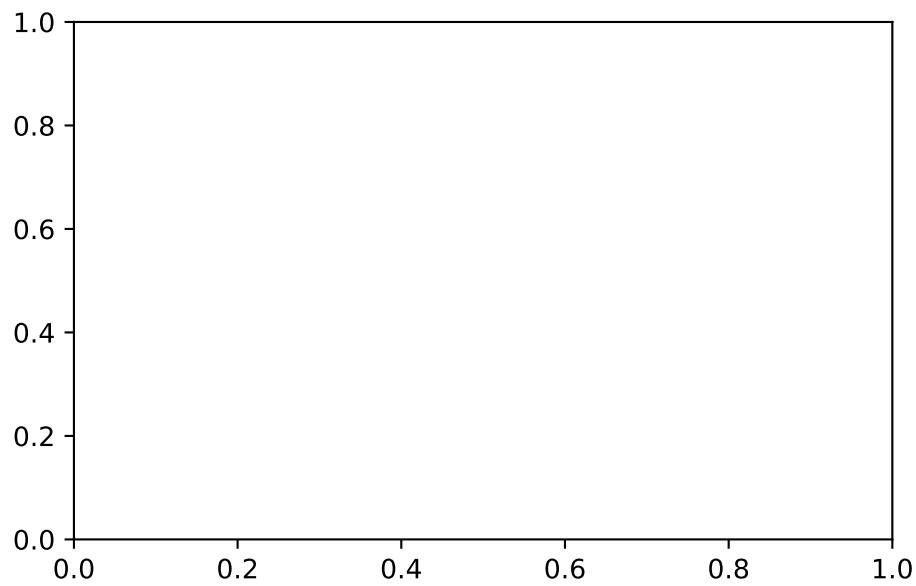
## 4 Figures and Axes

Two Matplotlib concepts should be distinguished:

- **Figure:** the complete graphical container;
- **Axes:** an individual plotting area within the figure.

A simple call:

```
1 fig, ax = plt.subplots()
```



creates both objects.

Their types can be inspected:

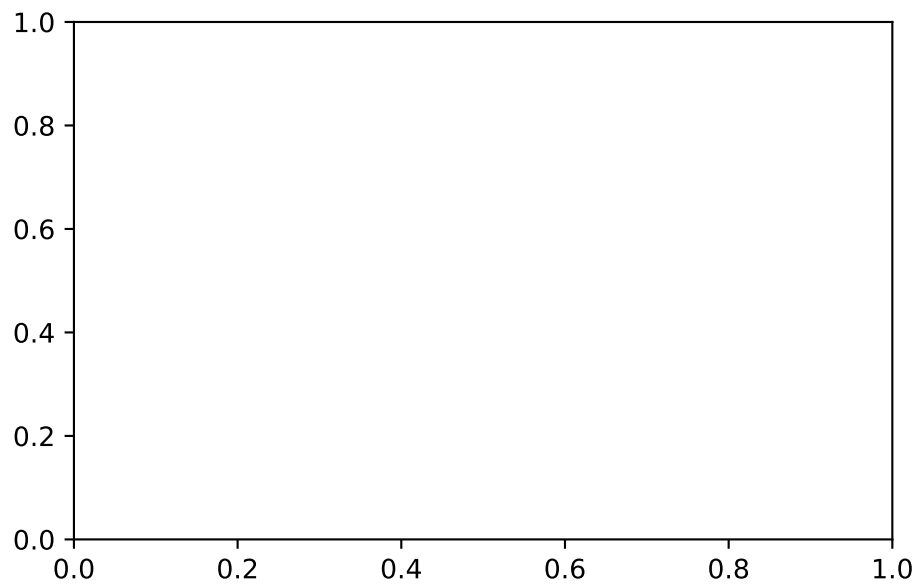
```
1 type(fig), type(ax)
```

```
(matplotlib.figure.Figure, matplotlib.axes._axes.Axes)
```

The Figure represents the complete output, while ax represents the coordinate system in which data are plotted.

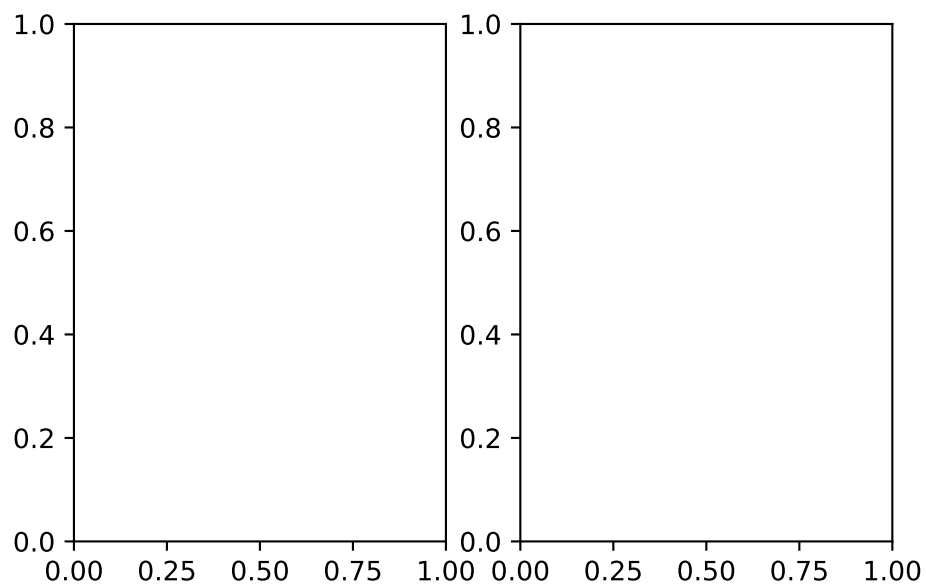
A single figure may contain one Axes:

```
1 fig, ax = plt.subplots()
```



or several:

```
1 fig, axes = plt.subplots(1, 2)
```



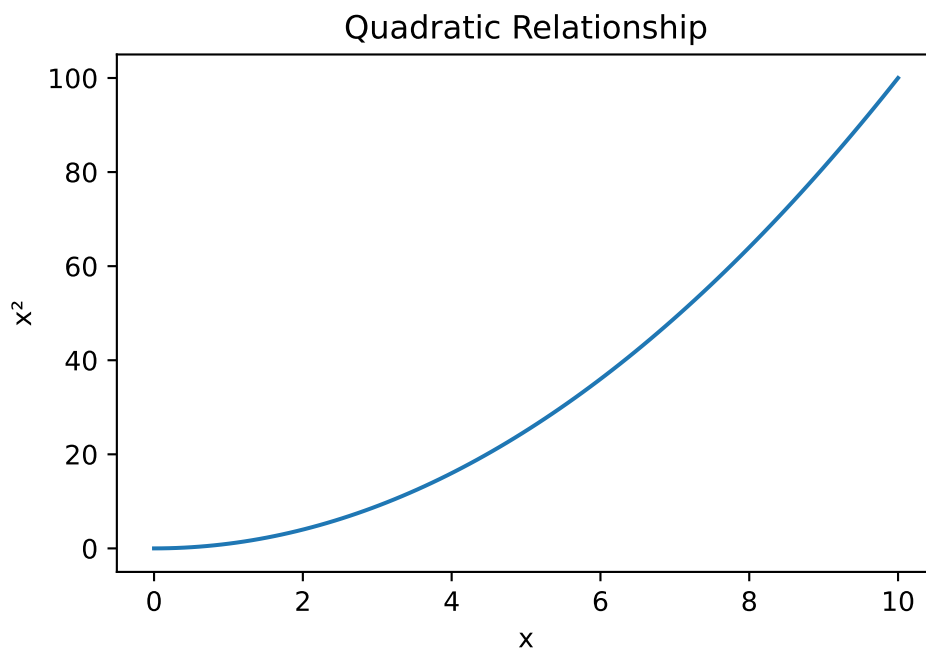
This distinction becomes important when constructing multiple related visualizations.

## 4.1 Adding Labels and Titles

A plot should communicate what its axes represent.

For example:

```
1 x = np.linspace(0, 10, 100)
2 y = x ** 2
3
4 fig, ax = plt.subplots()
5
6 ax.plot(x, y)
7
8 ax.set(
9     title="Quadratic Relationship",
10    xlabel="x",
11    ylabel="x2"
12 )
13
14 plt.show()
```



Titles and labels should provide information rather than simply repeat what is already visually obvious.

In applied work, labels should normally include units when those units are relevant.

For example:

```

1 ax.set(
2     xlabel="Height (cm)",
3     ylabel="Weight (kg)"
4 )

```

is more informative than:

```

1 ax.set(
2     xlabel="Height",
3     ylabel="Weight"
4 )

```

## 4.2 Legends

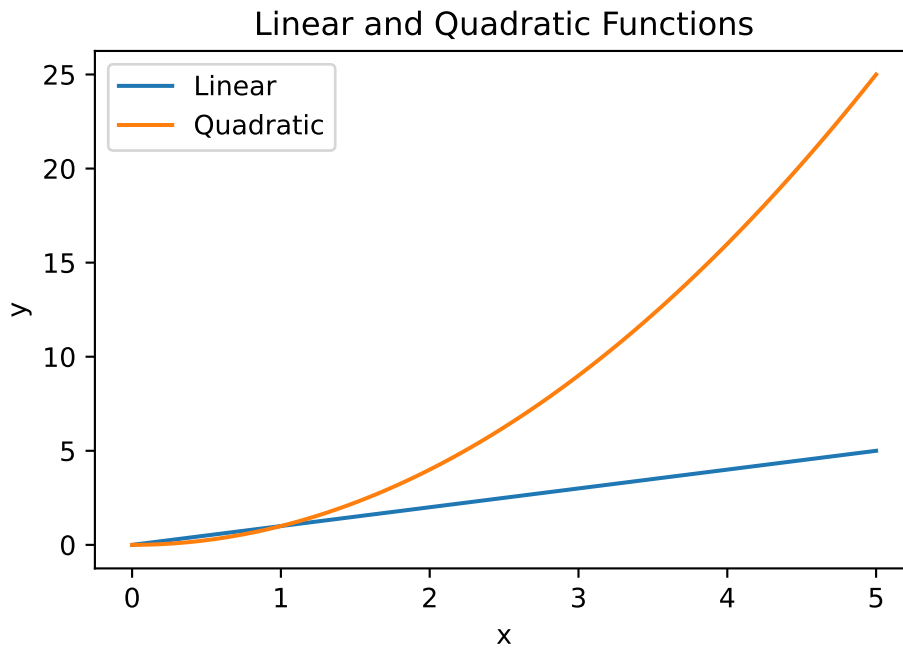
When several data series appear in the same Axes, labels can be associated with graphical elements:

```

1 x = np.linspace(0, 5, 100)
2
3 fig, ax = plt.subplots()
4
5 ax.plot(
6     x,
7     x,
8     label="Linear"
9 )
10
11 ax.plot(
12     x,
13     x ** 2,
14     label="Quadratic"
15 )
16
17 ax.set(
18     title="Linear and Quadratic Functions",
19     xlabel="x",
20     ylabel="y"
21 )
22

```

```
23 ax.legend()  
24  
25 plt.show()
```



A legend explains how graphical properties correspond to different groups or series.

A legend is useful only when the distinction it communicates is necessary. Redundant legends add visual complexity without additional information.

## 5 Scatter Plots

A **scatter plot** represents pairs of numerical observations as points in a two-dimensional coordinate system.

For observations

$$(x_i, y_i), \quad i = 1, \dots, n,$$

each observation is represented by one point whose horizontal and vertical coordinates are  $(x_i)$  and  $(y_i)$ .

Scatter plots are particularly useful for investigating:

- association between numerical variables;

- linear or nonlinear relationships;
- clusters;
- outliers;
- changes in variability;
- differences between groups.

Consider measurements of height and weight:

```
1 height = np.array([
2     174.3,
3     153.5,
4     162.1,
5     157.9,
6     174.8,
7     169.2,
8     172.0,
9     160.4,
10    152.8,
11    163.3
12 ])
13
14 weight = np.array([
15     65.7,
16     59.2,
17     57.3,
18     61.5,
19     77.3,
20     90.7,
21     85.4,
22     63.1,
23     73.2,
24     105.5
25 ])
```

A scatter plot can be created with:

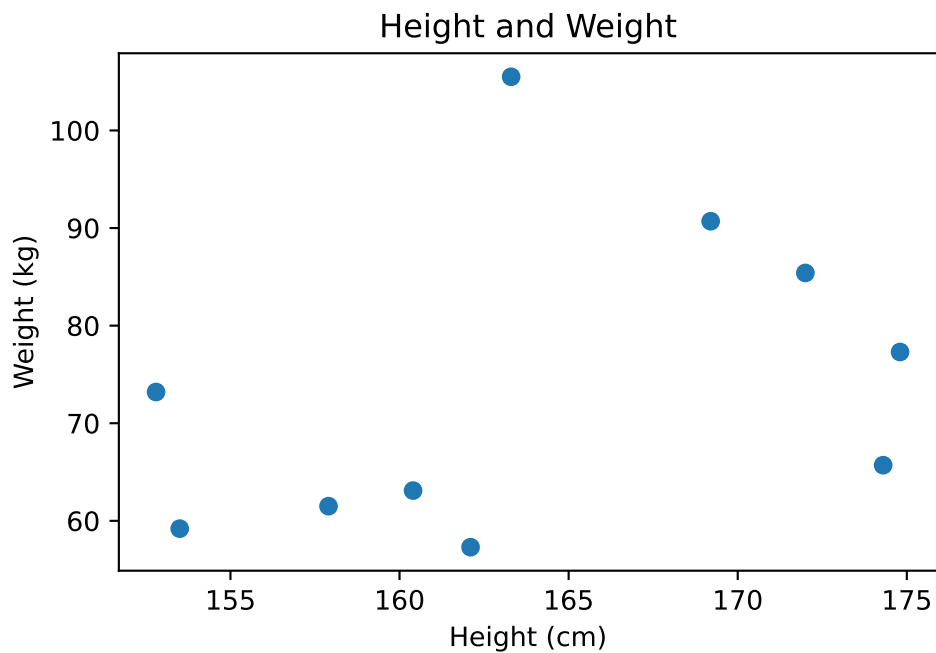
```
1 fig, ax = plt.subplots()
2
3 ax.scatter(
4     height,
5     weight
```



```

6 )
7
8 ax.set(
9     title="Height and Weight",
10    xlabel="Height (cm)",
11    ylabel="Weight (kg)"
12 )
13
14 plt.show()

```



The plot allows the relationship between the two variables to be inspected directly.

## 5.1 Association Is Not Causation

A scatter plot may provide evidence of an **association** between variables, but the visualization alone does not establish a causal relationship.

If larger values of (x) tend to occur with larger values of (y), we may observe a positive association.

If larger values of (x) tend to occur with smaller values of (y), we may observe a negative association.

However,

association  $\nRightarrow$  causation.

Causal conclusions require assumptions and evidence beyond the visual relationship itself.

This distinction is important when interpreting exploratory graphics.

## 5.2 Representing Groups

Several groups can be displayed in the same scatter plot.

For example:

```
1 height_a = np.array([
2     174.3, 153.5, 162.1, 157.9, 174.8,
3     169.2, 172.0, 160.4, 152.8, 163.3
4 ])
5
6 weight_a = np.array([
7     65.7, 59.2, 57.3, 61.5, 77.3,
8     90.7, 85.4, 63.1, 73.2, 105.5
9 ])
10
11 height_b = np.array([
12     154.2, 156.1, 160.3, 157.9, 162.6,
13     153.9, 170.1, 165.2, 157.6, 163.5
14 ])
15
16 weight_b = np.array([
17     55.3, 54.2, 47.3, 62.4, 77.5,
18     60.3, 52.4, 58.1, 50.2, 80.1
19 ])
```

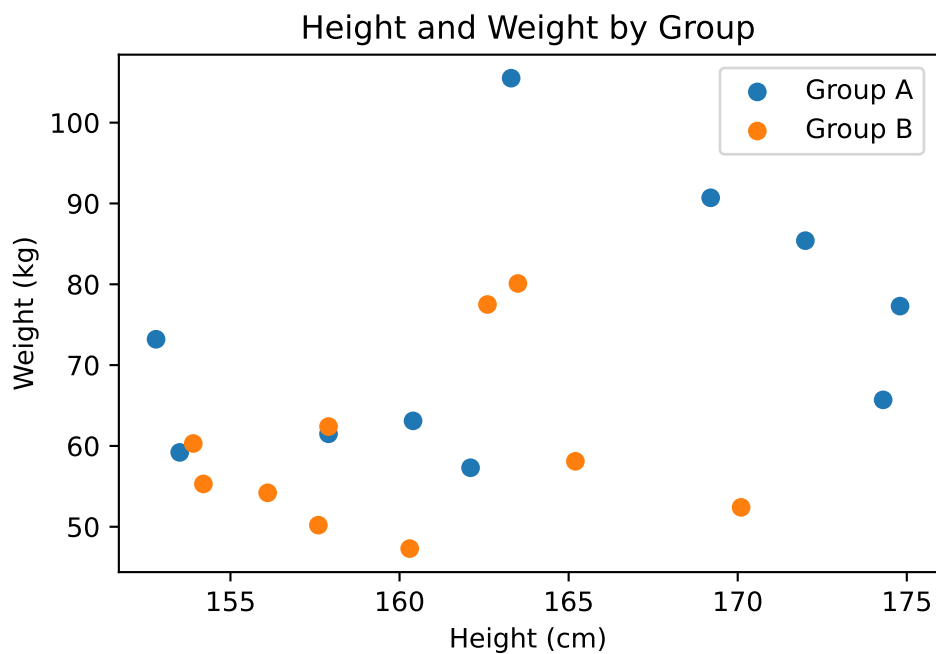
The groups can be represented separately:

```
1 fig, ax = plt.subplots()
2
3 ax.scatter(
4     height_a,
5     weight_a,
6     label="Group A"
```

```

7 )
8
9 ax.scatter(
10     height_b,
11     weight_b,
12     label="Group B"
13 )
14
15 ax.set(
16     title="Height and Weight by Group",
17     xlabel="Height (cm)",
18     ylabel="Weight (kg)"
19 )
20
21 ax.legend()
22
23 plt.show()

```



The visual distinction between groups makes it possible to examine both:

- the overall relationship between height and weight;
- potential differences between the groups.

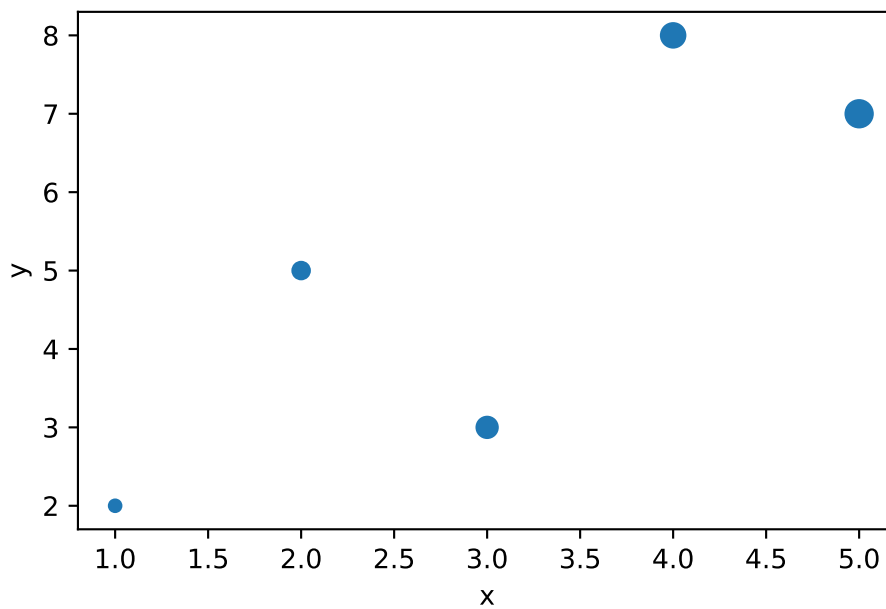
This is an example of using a graphical encoding to introduce another variable into the visualization.

### 5.3 Marker Size

Marker size can encode another numerical variable.

For example:

```
1 x = np.array([1, 2, 3, 4, 5])
2 y = np.array([2, 5, 3, 8, 7])
3 magnitude = np.array([20, 40, 60, 80, 100])
4
5 fig, ax = plt.subplots()
6
7 ax.scatter(
8     x,
9     y,
10    s=magnitude
11 )
12
13 ax.set(
14     xlabel="x",
15     ylabel="y"
16 )
17
18 plt.show()
```



The `s` parameter controls marker area.

Size should be used carefully because people generally compare positions more accurately than areas. If precise numerical comparison is important, position-based encodings are usually preferable.

## 5.4 Colour Maps

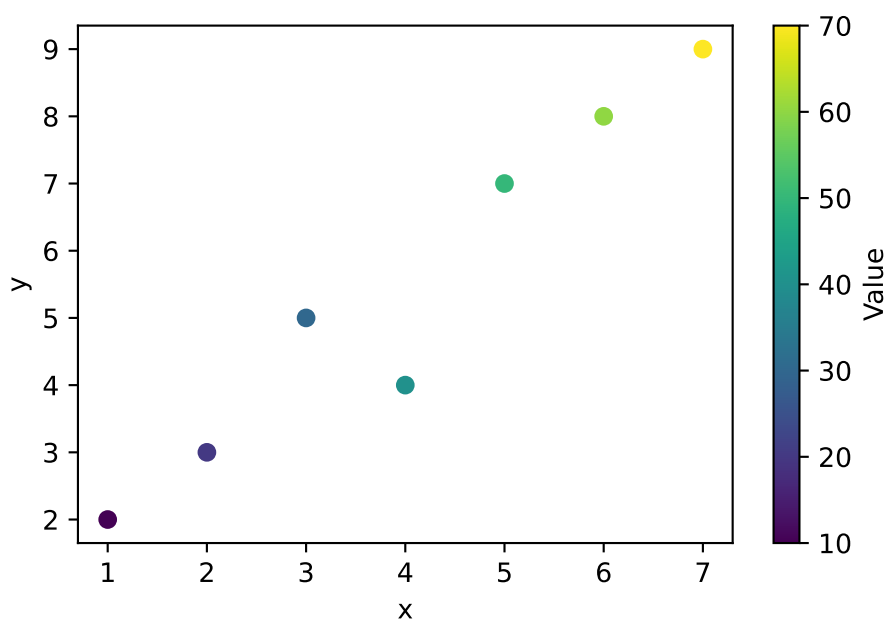
A continuous numerical variable can also be mapped to colour.

```
1 x = np.array([1, 2, 3, 4, 5, 6, 7])
2 y = np.array([2, 3, 5, 4, 7, 8, 9])
3 value = np.array([10, 20, 30, 40, 50, 60, 70])
4
5 fig, ax = plt.subplots()
6
7 scatter = ax.scatter(
8     x,
9     y,
10    c=value,
11    cmap="viridis"
12 )
13
14 fig.colorbar(
15     scatter,
```

```

16 ax=ax,
17 label="Value"
18 )
19
20 ax.set(
21     xlabel="x",
22     ylabel="y"
23 )
24
25 plt.show()

```



The colour bar communicates the mapping between colour and numerical value.

A colour map should be selected according to the meaning of the data. For example, sequential colour maps are generally appropriate when values progress from low to high, while diverging colour maps can be useful when values vary around a meaningful reference point.

## 5.5 Overplotting

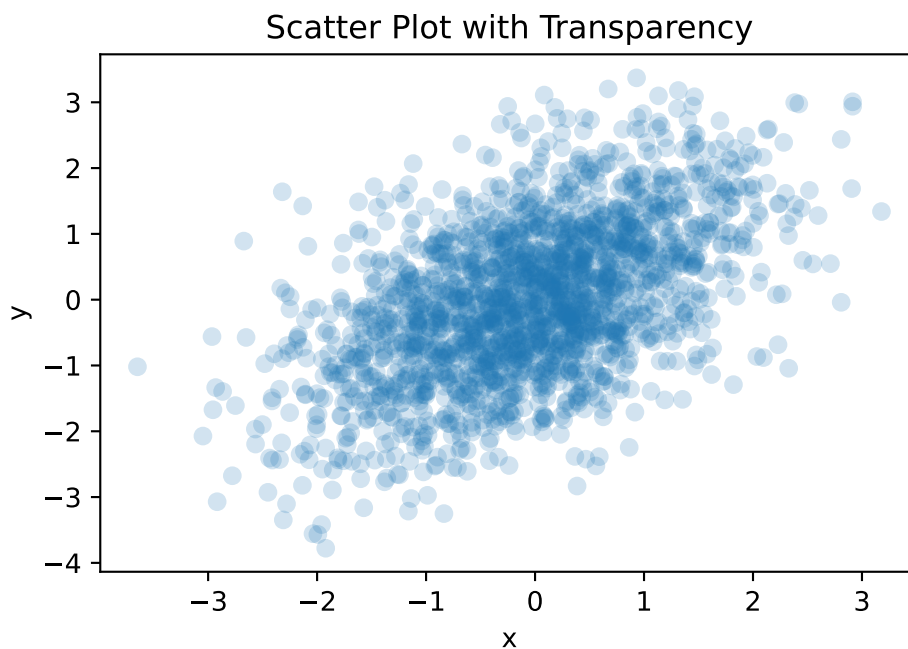
Scatter plots can become difficult to interpret when many observations occupy similar locations.

For example:

```

1  rng = np.random.default_rng(42)
2
3  x = rng.normal(size=2000)
4  y = 0.6 * x + rng.normal(size=2000)
5
6  fig, ax = plt.subplots()
7
8  ax.scatter(
9      x,
10     y,
11     alpha=0.2
12 )
13
14 ax.set(
15     title="Scatter Plot with Transparency",
16     xlabel="x",
17     ylabel="y"
18 )
19
20 plt.show()

```



Reducing opacity with alpha allows regions containing many overlapping observations to appear visually denser.

This does not solve every overplotting problem, but it illustrates an important principle: graphical parameters should be adjusted to improve the representation of the data, not merely for decoration.

## 6 Line Plots

A **line plot** connects ordered observations with line segments.

Consider observations

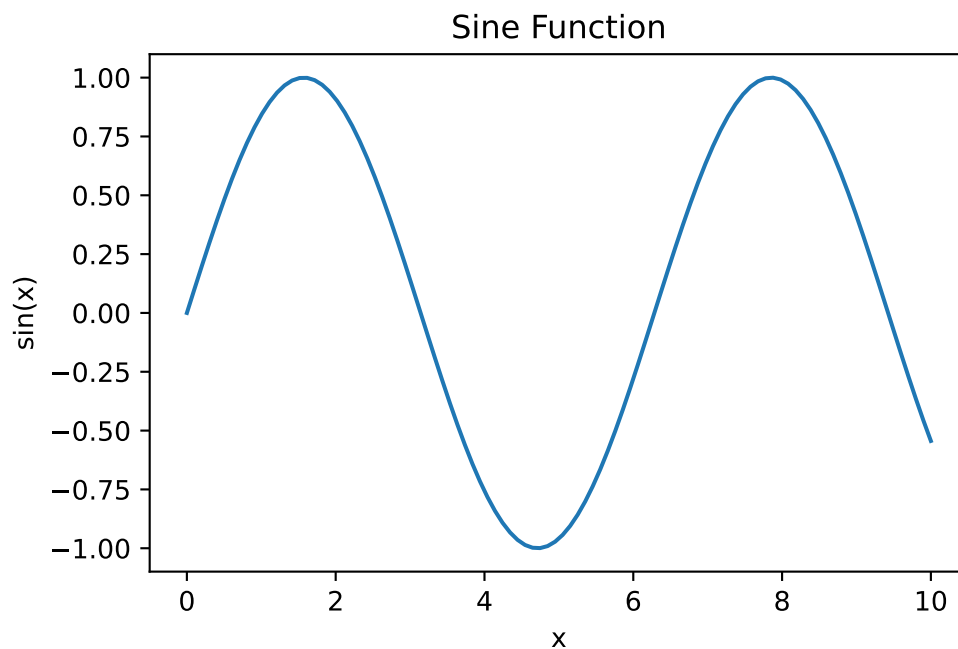
$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n).$$

A line plot connects consecutive observations according to their order.

For example:

```
1 x = np.linspace(0, 10, 100)
2 y = np.sin(x)
3
4 fig, ax = plt.subplots()
5
6 ax.plot(x, y)
7
8 ax.set(
9     title="Sine Function",
10    xlabel="x",
11    ylabel="sin(x)"
12 )
13
14 plt.show()
```





Line plots are particularly appropriate when the ordering of observations has meaning, such as:

- time;
- distance;
- iterations;
- model complexity;
- another continuous ordered variable.

## 6.1 Lines Imply Continuity or Ordered Progression

Connecting points with a line introduces an important visual implication: values between neighbouring observations are presented as part of an ordered progression.

For this reason, line plots should not be used merely because points can be connected.

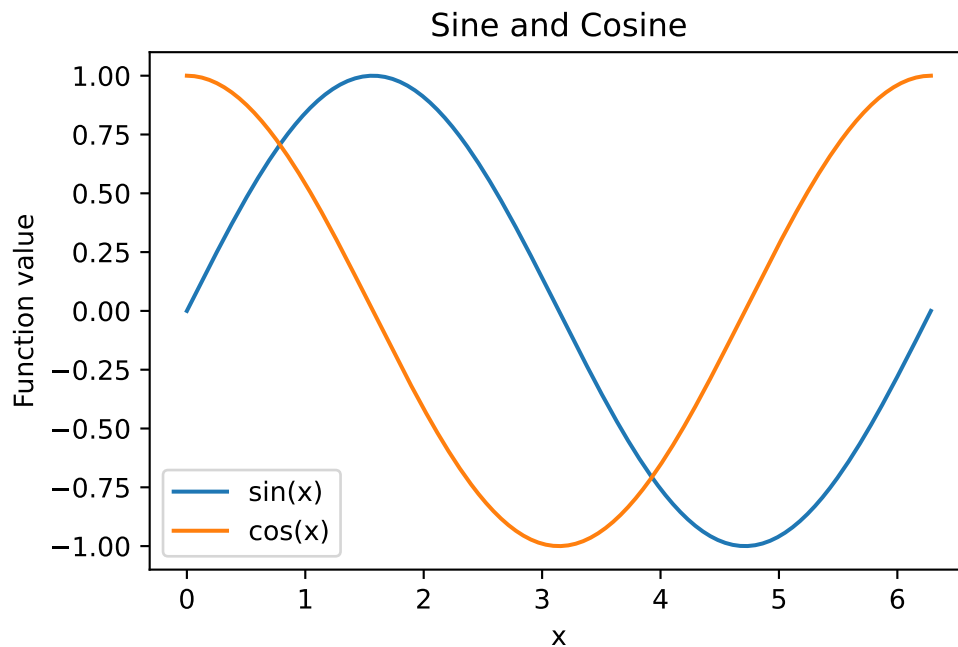
Suppose observations correspond to unrelated categories. Connecting them with a line may visually imply continuity that does not exist.

A line plot is most natural when the horizontal variable has an inherent ordering and the connection between consecutive observations is meaningful.

## 6.2 Multiple Series

Several related series can be represented on the same Axes.

```
1 x = np.linspace(0, 2 * np.pi, 200)
2
3 fig, ax = plt.subplots()
4
5 ax.plot(
6     x,
7     np.sin(x),
8     label="sin(x)"
9 )
10
11 ax.plot(
12     x,
13     np.cos(x),
14     label="cos(x)"
15 )
16
17 ax.set(
18     title="Sine and Cosine",
19     xlabel="x",
20     ylabel="Function value"
21 )
22
23 ax.legend()
24
25 plt.show()
```



This allows the functions to be compared using a common coordinate system.

However, adding more series is not always better. As the number of lines increases, overlap and legend complexity can make the visualization difficult to interpret.

In such cases, separate panels or another representation may communicate the information more effectively.

### 6.3 Line Style and Markers

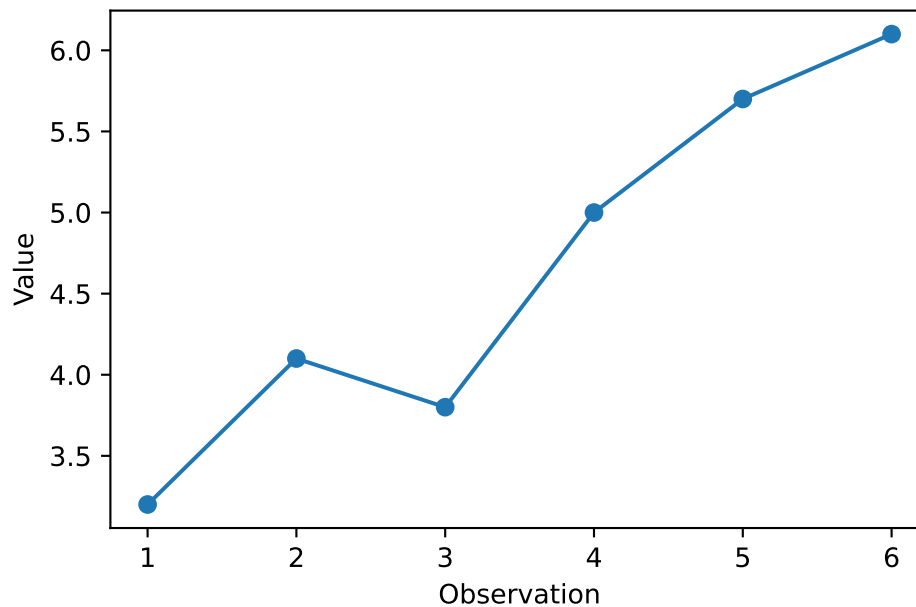
A line can also include markers representing the actual observed values:

```
1 x = np.arange(1, 7)
2 y = np.array([3.2, 4.1, 3.8, 5.0, 5.7, 6.1])
3
4 fig, ax = plt.subplots()
5
6 ax.plot(
7     x,
8     y,
9     marker="o"
10 )
11
12 ax.set(
```

```

13     xlabel="Observation",
14     ylabel="Value"
15 )
16
17 plt.show()

```



Markers make the locations of observed values explicit, while the line emphasizes their progression.

This can be useful when relatively few observations are available.

## 6.4 Mathematical Functions

Line plots are also useful for visualizing mathematical functions.

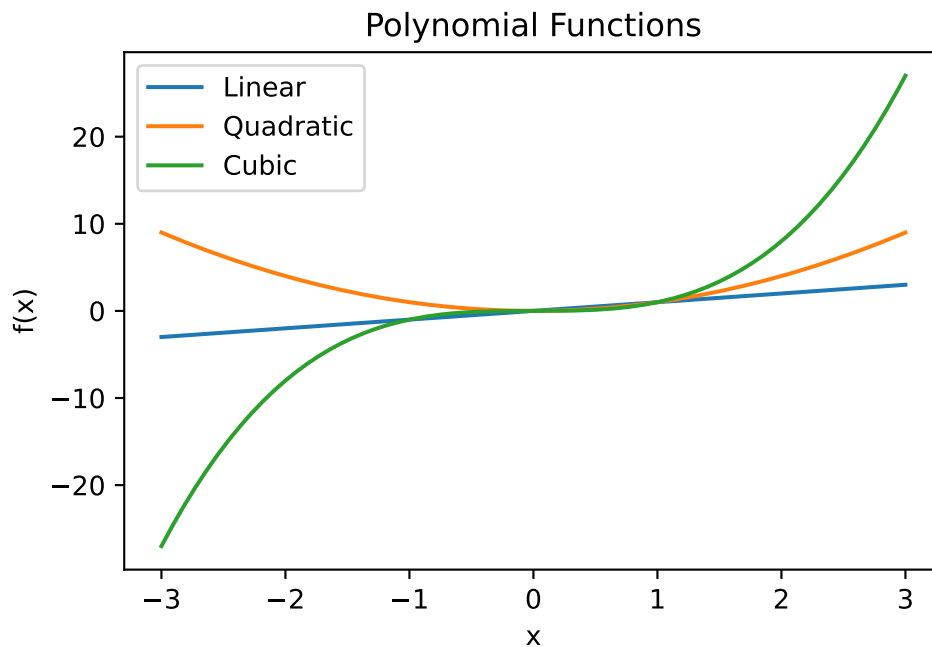
For example:

```

1 x = np.linspace(-3, 3, 200)
2
3 y_linear = x
4 y_quadratic = x ** 2
5 y_cubic = x ** 3
6
7 fig, ax = plt.subplots()
8

```

```
9  ax.plot(  
10      x,  
11      y_linear,  
12      label="Linear"  
13  )  
14  
15  ax.plot(  
16      x,  
17      y_quadratic,  
18      label="Quadratic"  
19  )  
20  
21  ax.plot(  
22      x,  
23      y_cubic,  
24      label="Cubic"  
25  )  
26  
27  ax.set(  
28      title="Polynomial Functions",  
29      xlabel="x",  
30      ylabel="f(x)"  
31  )  
32  
33  ax.legend()  
34  
35  plt.show()
```



This type of representation is useful when studying mathematical objects that later appear in statistical and machine learning models.

For example, visualizing a loss function, activation function, regularization path, or learning curve can make its numerical behaviour easier to understand.

## 6.5 Scatter Plot or Line Plot?

The distinction between scatter and line plots is conceptual rather than merely stylistic.

A scatter plot primarily asks:

**How are two numerical variables related across observations?**

A line plot primarily asks:

**How does a quantity change along an ordered dimension?**

For cross-sectional observations with no meaningful sequence, a scatter plot is generally more natural.

For temporal observations, iterations, or values of a continuous mathematical function, a line plot is often more appropriate.

The choice of plot should therefore follow the structure of the analytical question rather than personal graphical preference.

## 7 Bar Plots

A **bar plot** represents numerical values using the lengths of rectangular bars.

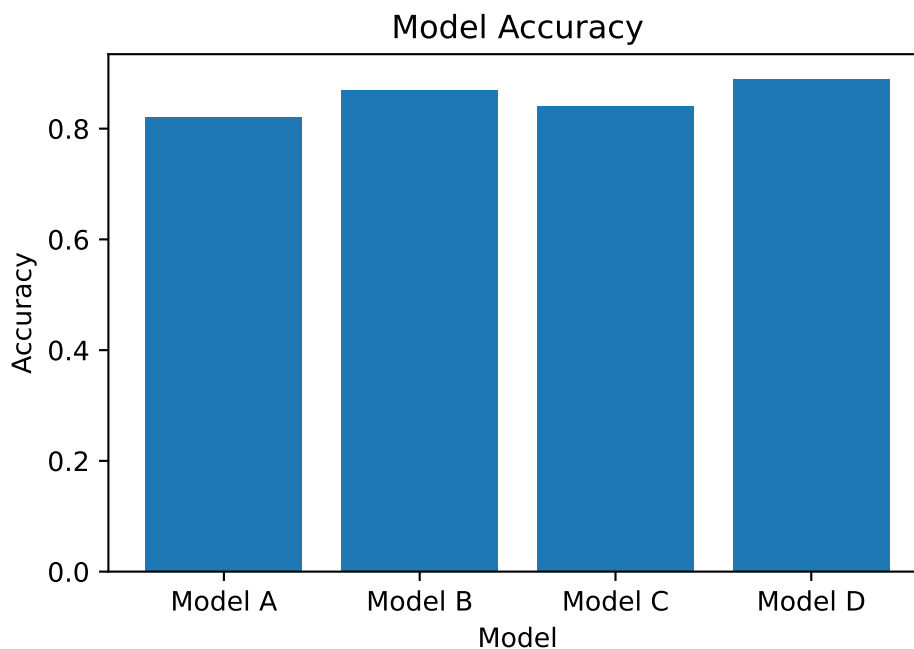
Unlike scatter and line plots, which usually represent relationships between numerical variables, bar plots are particularly useful when one variable identifies **discrete categories** and another represents a numerical quantity associated with each category.

Consider:

```
1 categories = np.array([
2     "Model A",
3     "Model B",
4     "Model C",
5     "Model D"
6 ])
7
8 accuracy = np.array([
9     0.82,
10    0.87,
11    0.84,
12    0.89
13 ])
```

A bar plot can be created with:

```
1 fig, ax = plt.subplots()
2
3 ax.bar(
4     categories,
5     accuracy
6 )
7
8 ax.set(
9     title="Model Accuracy",
10    xlabel="Model",
11    ylabel="Accuracy"
12 )
13
14 plt.show()
```



The height of each bar encodes the numerical value associated with the corresponding category.

Mathematically, if category ( $c_i$ ) is associated with value ( $y_i$ ), the graphical mapping is:

$$c_i \longrightarrow \text{horizontal position}$$

and:

$$y_i \longrightarrow \text{bar length.}$$

The categories themselves do not need to represent numerical values or have meaningful distances between them.

## 7.1 Bar Plots and Aggregated Data

Bar plots frequently represent **aggregated quantities** rather than individual observations.

Suppose a dataset contains observations belonging to three groups. A bar plot might represent:

- the number of observations in each group;
- the mean value within each group;
- total revenue by category;

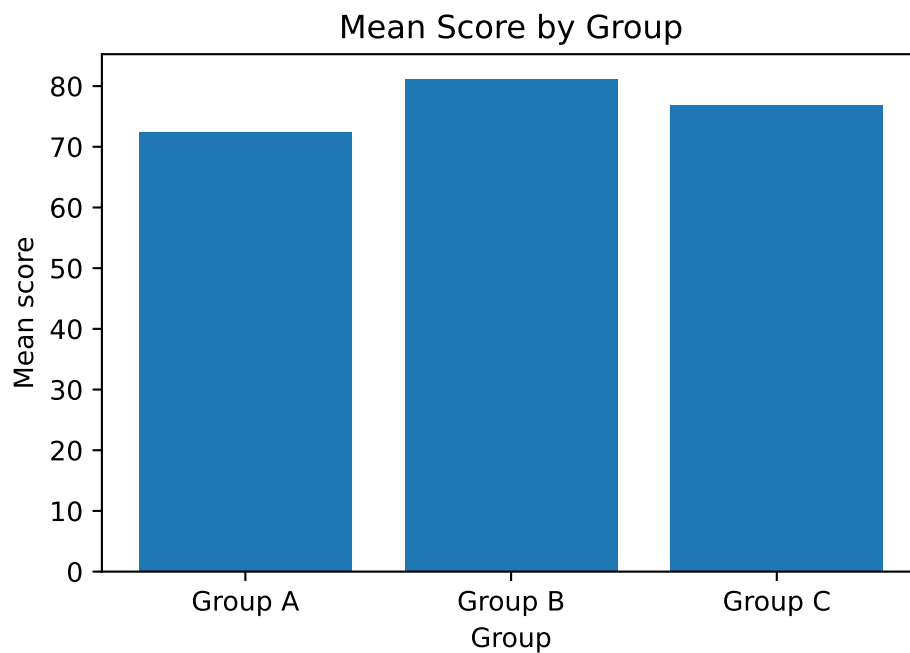


- error rate by model;
- frequency of each class.

This distinction matters because the viewer should understand what each bar actually represents.

For example:

```
1 groups = np.array([
2     "Group A",
3     "Group B",
4     "Group C"
5 ])
6
7 mean_score = np.array([
8     72.5,
9     81.2,
10    76.8
11 ])
12
13 fig, ax = plt.subplots()
14
15 ax.bar(
16     groups,
17     mean_score
18 )
19
20 ax.set(
21     title="Mean Score by Group",
22     xlabel="Group",
23     ylabel="Mean score"
24 )
25
26 plt.show()
```



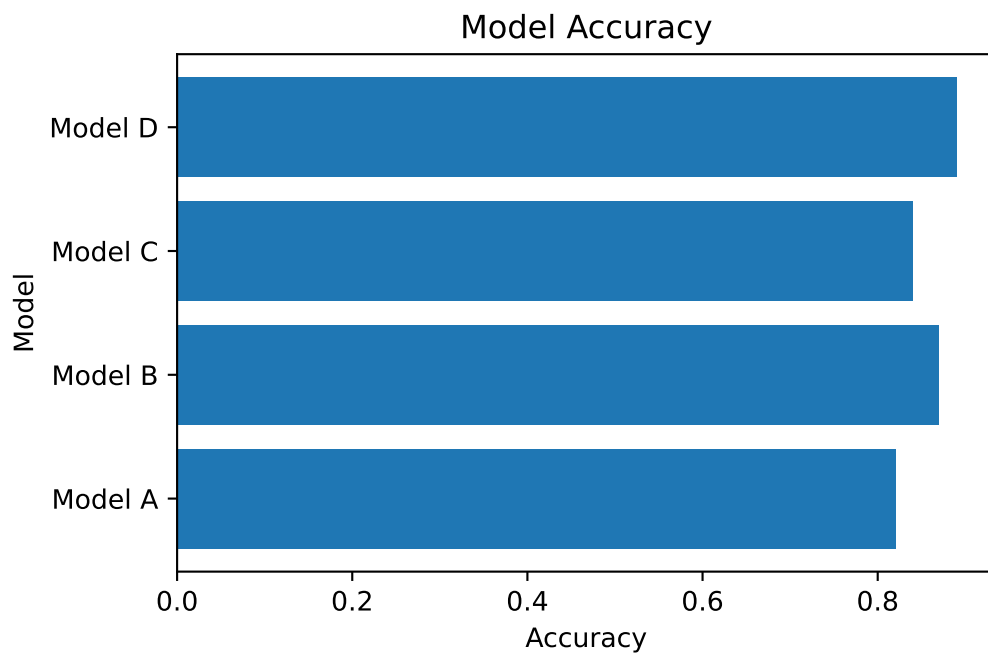
The bars represent **group means**, not individual observations.

Whenever a visualization displays an aggregate, interpretation depends on understanding how that aggregate was constructed.

## 7.2 Horizontal Bar Plots

Bars can also be displayed horizontally:

```
1 fig, ax = plt.subplots()
2
3 ax.barh(
4     categories,
5     accuracy
6 )
7
8 ax.set(
9     title="Model Accuracy",
10    xlabel="Accuracy",
11    ylabel="Model"
12 )
13
14 plt.show()
```



Horizontal bar plots can be particularly useful when category labels are long or when many categories must be displayed.

### 7.3 Ordering Categories

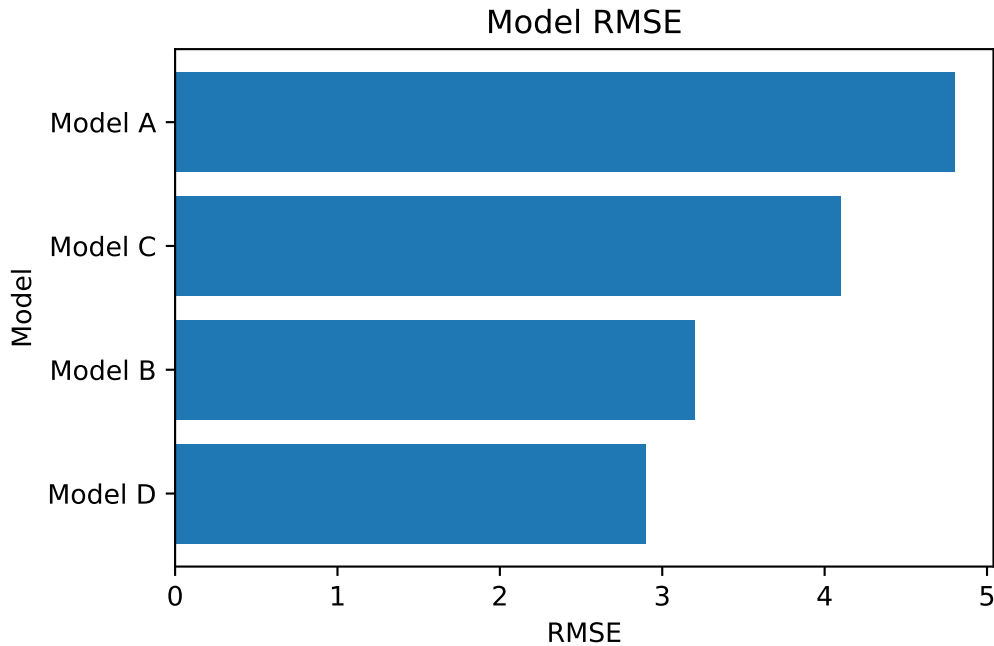
The order of bars can influence how easily differences are perceived.

Suppose:

```
1 models = np.array([
2     "Model A",
3     "Model B",
4     "Model C",
5     "Model D"
6 ])
7
8 rmse = np.array([
9     4.8,
10    3.2,
11    4.1,
12    2.9
13 ])
```

If the categories have no inherent ordering, sorting by the numerical quantity can improve comparison:

```
1 order = np.argsort(rmse)
2
3 fig, ax = plt.subplots()
4
5 ax.barh(
6     models[order],
7     rmse[order]
8 )
9
10 ax.set(
11     title="Model RMSE",
12     xlabel="RMSE",
13     ylabel="Model"
14 )
15
16 plt.show()
```



However, sorting is not always appropriate. Categories may already possess a meaningful natural order, such as:

- age groups;

- educational levels;
- stages of a process;
- ordered risk categories.

Graphical ordering should preserve meaningful structure when that structure exists.

## 7.4 Baselines and Bar Length

Bar plots encode values through **length**. For this reason, the baseline is particularly important.

Consider a comparison between values:

95      and      100.

If the vertical axis begins near 94 rather than zero, the graphical difference between the bars can appear much larger than the numerical difference.

For ordinary bar charts, a zero baseline is therefore generally appropriate because the length of each bar should remain proportional to the represented value.

This differs from line plots, where a non-zero axis range can sometimes be appropriate when the purpose is to examine small variations over a continuous scale.

The appropriate axis design depends on the graphical encoding being used.

## 8 Histograms

A **histogram** represents the distribution of a numerical variable by dividing its range into intervals called **bins** and counting how many observations fall within each interval.

Consider a sample generated from a normal distribution:

```

1 rng = np.random.default_rng(42)
2
3 values = rng.normal(
4     loc=0,
5     scale=1,
6     size=1000
7 )

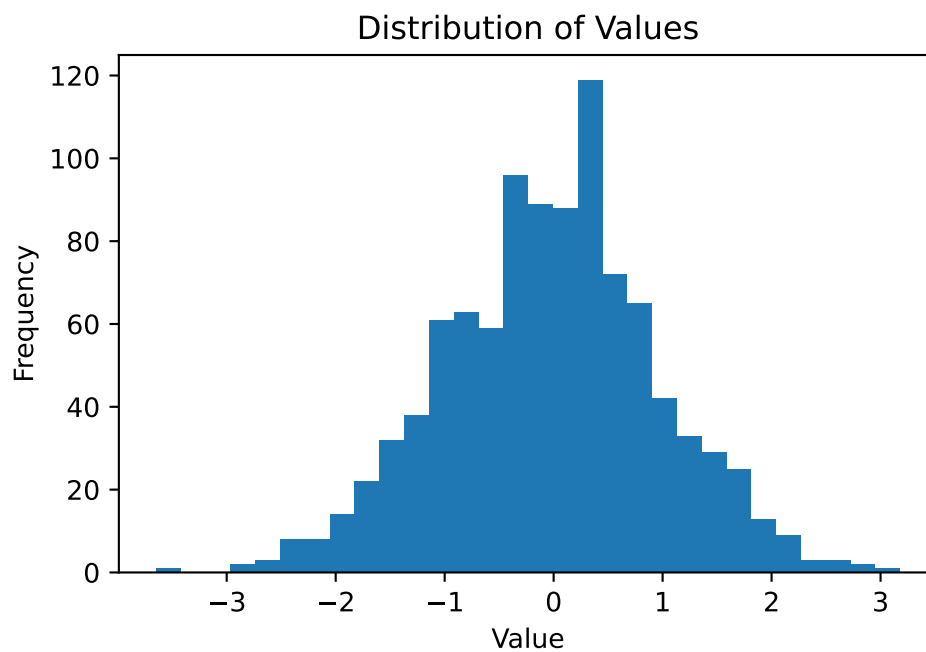
```

A histogram can be created with:

```

1 fig, ax = plt.subplots()
2
3 ax.hist(
4     values,
5     bins=30
6 )
7
8 ax.set(
9     title="Distribution of Values",
10    xlabel="Value",
11    ylabel="Frequency"
12 )
13
14 plt.show()

```



Unlike a bar plot, the horizontal axis of a histogram represents a **continuous numerical variable**.

The bars correspond to adjacent numerical intervals rather than independent categories.

## 8.1 Bins

Suppose the range of a numerical variable is divided into intervals:

$$B_1, B_2, \dots, B_k.$$

For each bin ( $B_j$ ), the histogram counts:

$$n_j = \sum_{i=1}^n \mathbf{1}(x_i \in B_j),$$

where  $\mathbf{1}(\cdot)$  is an indicator function.

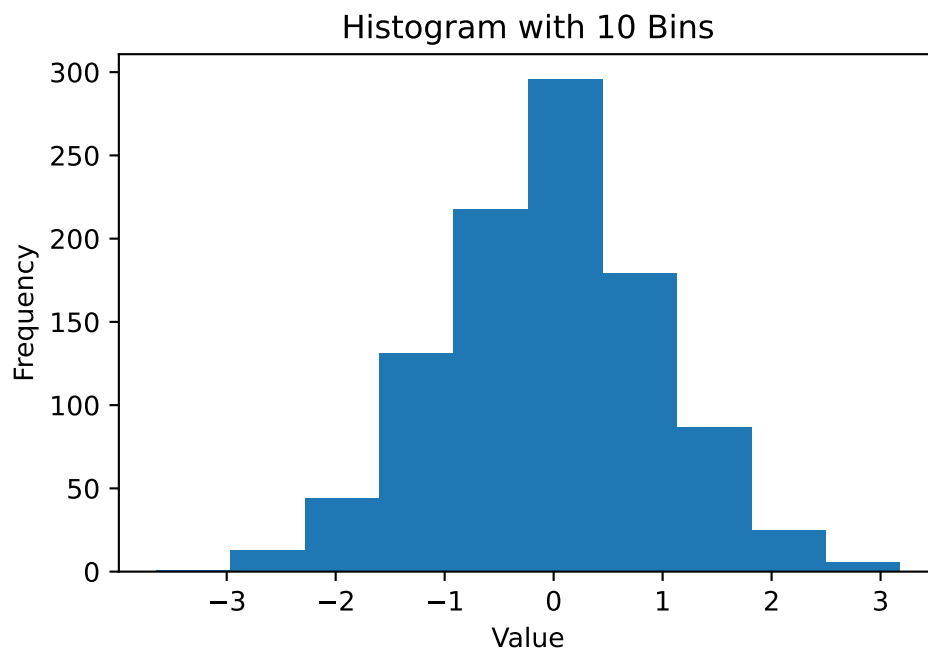
The height of the ( $j$ )-th bar therefore represents the number of observations assigned to that interval.

The choice of bins affects the appearance of the histogram.

For example:

```

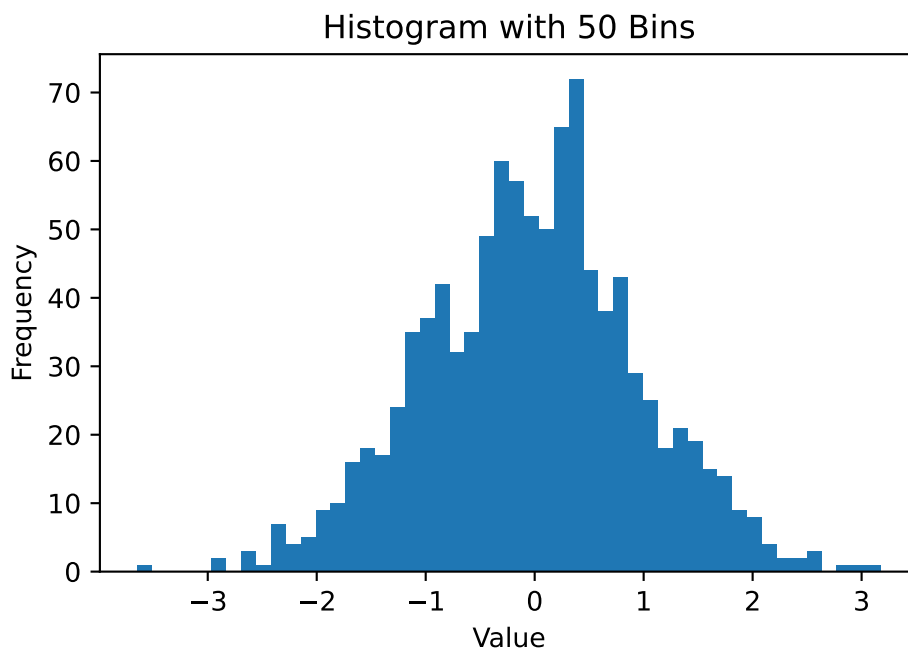
1 fig, ax = plt.subplots()
2
3 ax.hist(
4     values,
5     bins=10
6 )
7
8 ax.set(
9     title="Histogram with 10 Bins",
10    xlabel="Value",
11    ylabel="Frequency"
12 )
13
14 plt.show()
```



and:

```
1 fig, ax = plt.subplots()
2
3 ax.hist(
4     values,
5     bins=50
6 )
7
8 ax.set(
9     title="Histogram with 50 Bins",
10    xlabel="Value",
11    ylabel="Frequency"
12 )
13
14 plt.show()
```





represent exactly the same observations but emphasize different levels of detail.

Too few bins can conceal important structure.

Too many bins can emphasize random sample variation and make the distribution appear unnecessarily irregular.

The binning strategy is therefore part of the statistical representation, not merely a cosmetic parameter.

## 8.2 What a Histogram Can Reveal

Histograms can help identify properties such as:

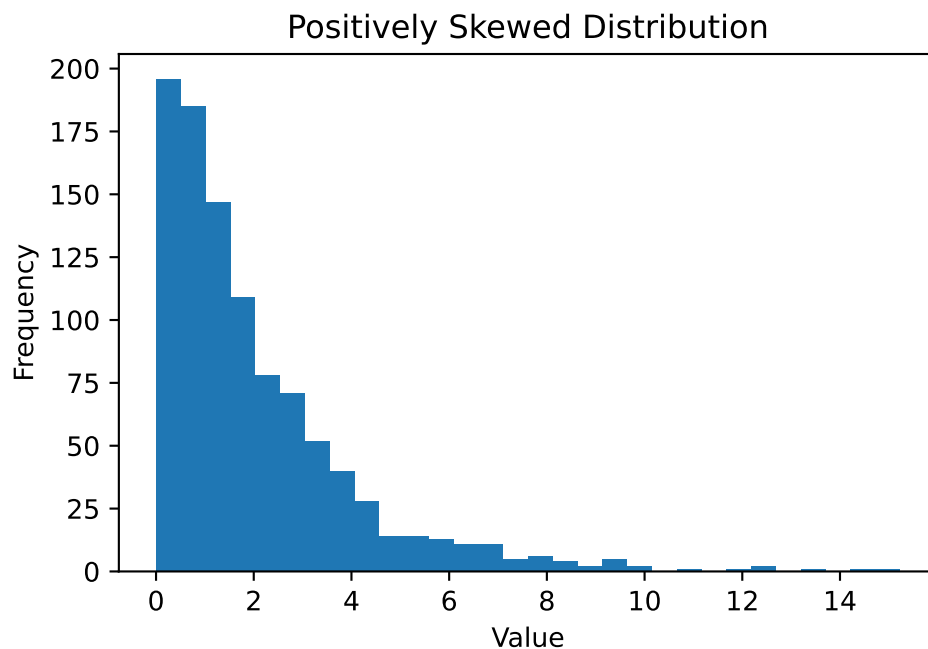
- central tendency;
- spread;
- skewness;
- multiple modes;
- heavy tails;
- potential outliers;
- gaps in the observed values.

For example, a positively skewed distribution might be generated as:

```

1 values = rng.exponential(
2     scale=2,
3     size=1000
4 )
5
6 fig, ax = plt.subplots()
7
8 ax.hist(
9     values,
10    bins=30
11 )
12
13 ax.set(
14     title="Positively Skewed Distribution",
15     xlabel="Value",
16     ylabel="Frequency"
17 )
18
19 plt.show()

```



The asymmetry of the distribution becomes visually apparent.

A histogram therefore provides information that cannot generally be recovered from a mean and standard deviation alone.

### 8.3 Frequency and Density

A histogram can represent either counts or density.

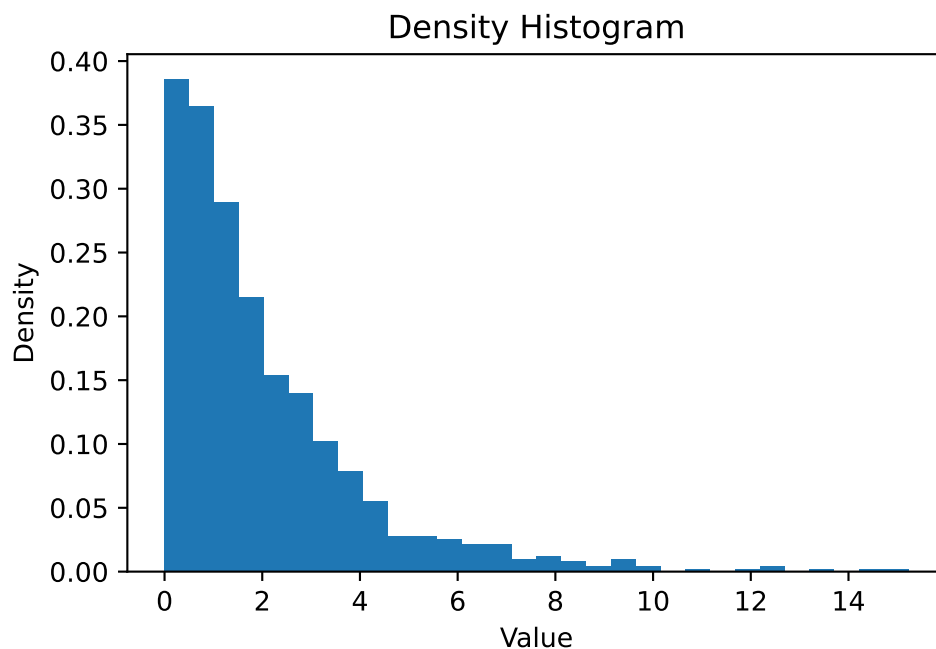
By default:

```
1 ax.hist(values)
```

represents frequencies.

Using:

```
1 fig, ax = plt.subplots()
2
3 ax.hist(
4     values,
5     bins=30,
6     density=True
7 )
8
9 ax.set(
10     title="Density Histogram",
11     xlabel="Value",
12     ylabel="Density"
13 )
14
15 plt.show()
```



normalizes the histogram so that the total area is approximately:

1.

A density histogram is particularly useful when comparing an empirical distribution with a probability density function.

It is important to distinguish **density** from **probability**. The height of a density histogram is not generally the probability of observing one particular value. Probabilities correspond to areas over intervals.

## 9 Box Plots

A **box plot** provides a compact representation of the distribution of a numerical variable.

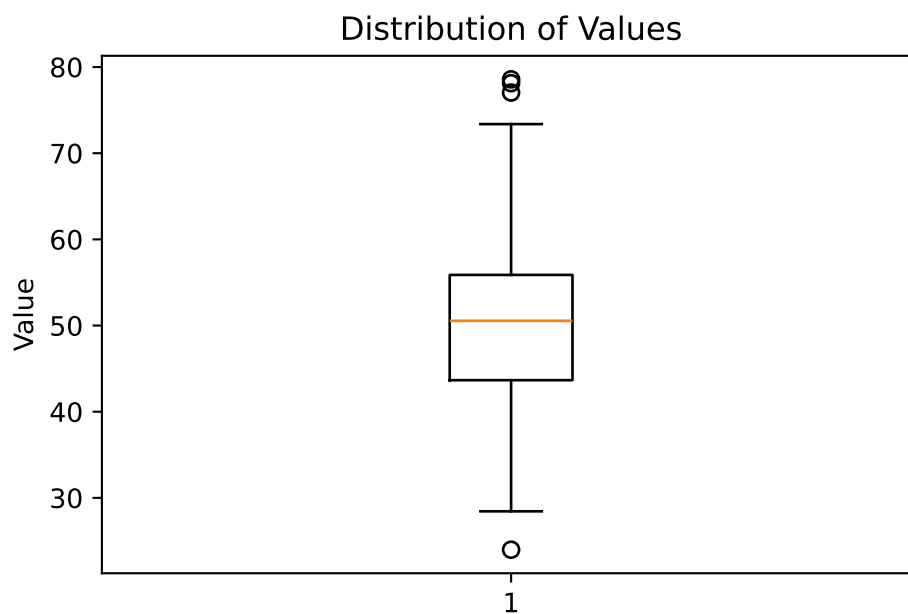
Consider:

```
1 values = rng.normal(  
2     loc=50,  
3     scale=10,  
4     size=200  
5 )  
6
```

```

7 fig, ax = plt.subplots()
8
9 ax.boxplot(values)
10
11 ax.set(
12     title="Distribution of Values",
13     ylabel="Value"
14 )
15
16 plt.show()

```



A conventional box plot summarizes several aspects of the distribution using quartiles.

Let:

$$Q_1$$

denote the first quartile,

$$Q_2$$

the median, and:

$$Q_3$$

the third quartile.

The **interquartile range** is:

$$\text{IQR} = Q_3 - Q_1. \quad (1)$$

The box extends from ( $Q_1$ ) to ( $Q_3$ ), with the median represented inside the box.

## 9.1 Whiskers and Potential Outliers

A common box-plot convention uses boundaries based on:

$$Q_1 - 1.5 \text{ IQR}$$

and:

$$Q_3 + 1.5 \text{ IQR}.$$

Observations beyond these boundaries may be displayed individually.

These points are often described as **potential outliers**, but they should not automatically be interpreted as errors or invalid observations.

An observation can be unusual while still being legitimate.

Whether an extreme value represents:

- measurement error;
- data-entry error;
- a rare but valid observation;
- a genuinely important event

requires domain knowledge and further investigation.

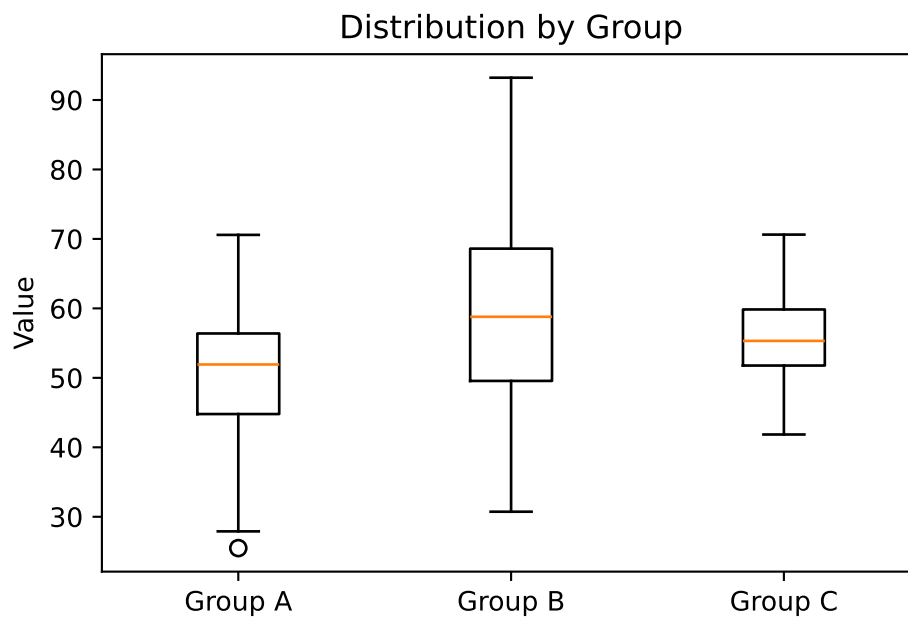
The box plot identifies observations that deserve attention; it does not determine their substantive meaning.

## 9.2 Comparing Groups

Box plots are particularly useful for comparing distributions across categories.

For example:

```
1 group_a = rng.normal(  
2     loc=50,  
3     scale=8,  
4     size=100  
5 )  
6  
7 group_b = rng.normal(  
8     loc=60,  
9     scale=12,  
10    size=100  
11 )  
12  
13 group_c = rng.normal(  
14     loc=55,  
15     scale=6,  
16     size=100  
17 )  
18  
19 fig, ax = plt.subplots()  
20  
21 ax.boxplot(  
22     [group_a, group_b, group_c],  
23     tick_labels=[  
24         "Group A",  
25         "Group B",  
26         "Group C"  
27     ]  
28 )  
29  
30 ax.set(  
31     title="Distribution by Group",  
32     ylabel="Value"  
33 )  
34  
35 plt.show()
```



The plot allows the groups to be compared in terms of:

- median;
- interquartile range;
- overall spread;
- asymmetry;
- potential extreme observations.

However, a box plot deliberately compresses information. It does not show the complete shape of the distribution.

For this reason, box plots and histograms answer related but different questions.

## 10 Bar Plot, Histogram, or Box Plot?

These graphical forms should not be selected interchangeably.

A **bar plot** is appropriate when comparing numerical quantities across discrete categories.

A **histogram** is appropriate when examining the shape of the distribution of a numerical variable.

A **box plot** is appropriate when a compact summary of one or several numerical distributions is useful.

The underlying questions differ:



**Bar plot:** How does a numerical quantity differ across categories?

**Histogram:** How are observations distributed across the range of a numerical variable?

**Box plot:** How do location, spread, and potential extreme values compare across distributions?

Selecting a visualization begins with identifying which of these questions is being asked.

## 11 Multiple Plots and Subplots

A single figure can contain multiple Axes.

This is useful when several related graphical representations should be compared without placing all graphical elements in the same coordinate system.

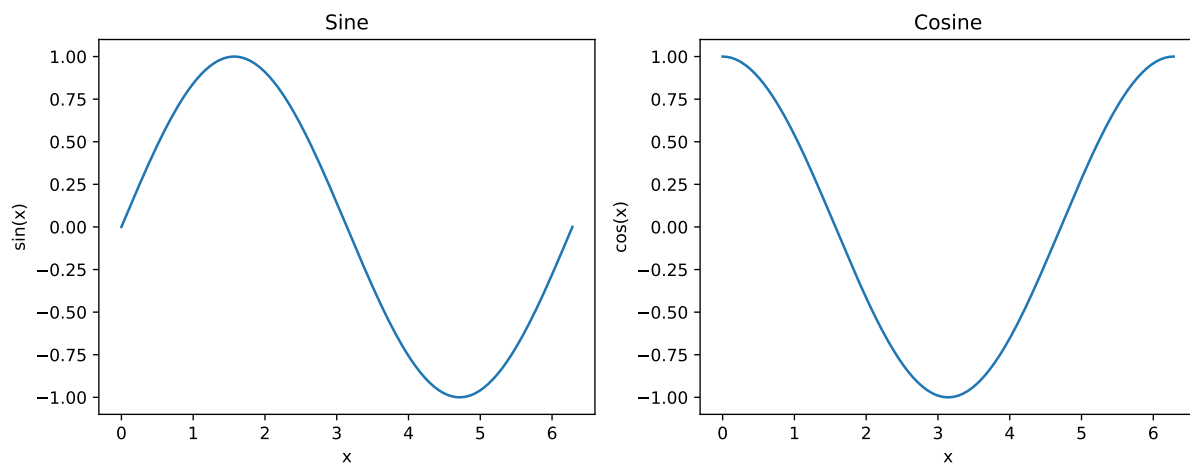
For example:

```
1 x = np.linspace(  
2     0,  
3     2 * np.pi,  
4     200  
5 )  
6  
7 fig, axes = plt.subplots(  
8     1,  
9     2,  
10    figsize=(10, 4)  
11 )  
12  
13 axes[0].plot(  
14     x,  
15     np.sin(x)  
16 )  
17  
18 axes[0].set(  
19     title="Sine",  
20     xlabel="x",  
21     ylabel="sin(x)"  
22 )  
23
```

```

24 axes[1].plot(
25     x,
26     np.cos(x)
27 )
28
29 axes[1].set(
30     title="Cosine",
31     xlabel="x",
32     ylabel="cos(x)"
33 )
34
35 fig.tight_layout()
36
37 plt.show()

```



Here:

```

1 fig

```

represents the complete figure, while:

```

1 axes[0]
2 axes[1]

```

represent separate plotting areas.

## 11.1 Two-Dimensional Axes Arrays

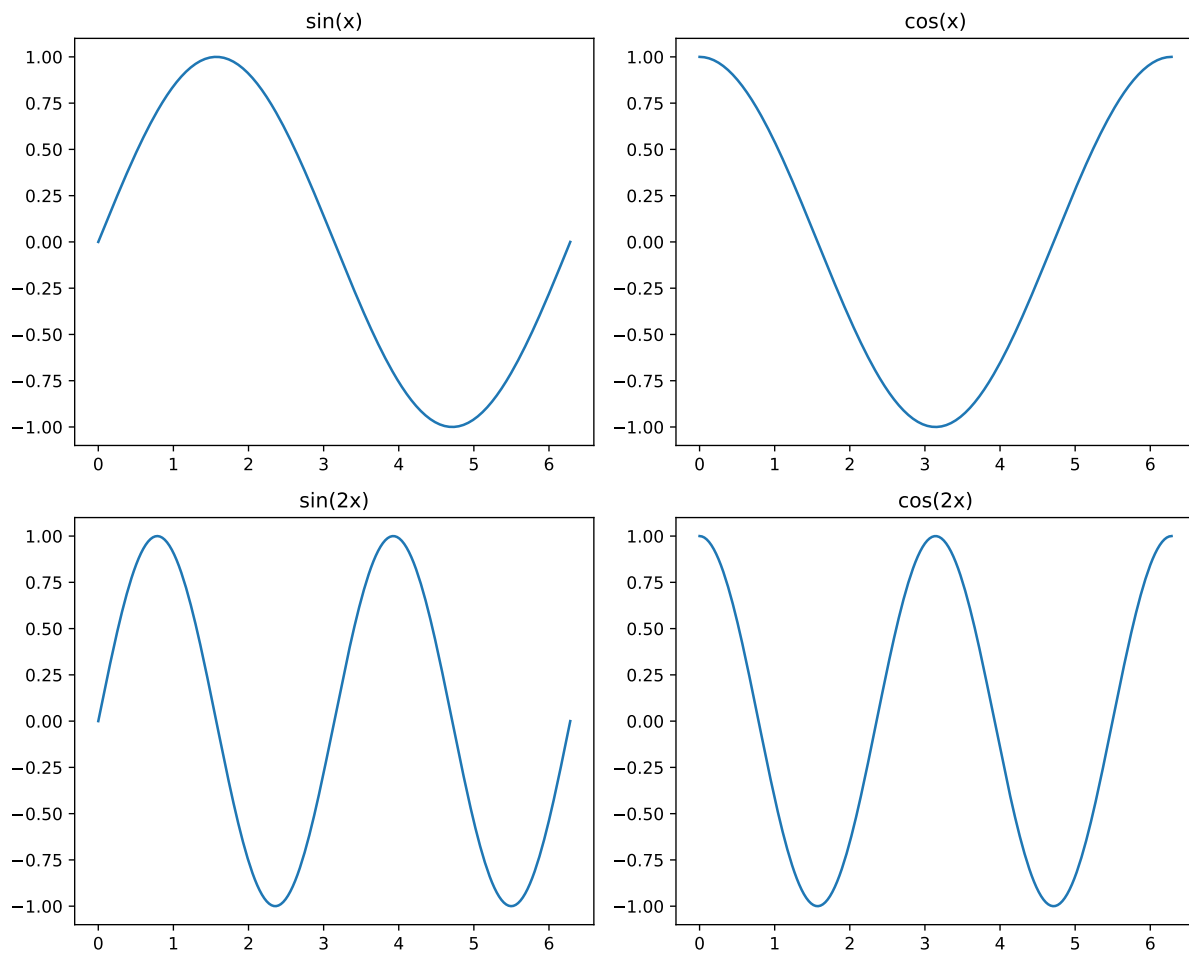
A grid of plots can also be created:

```
1 fig, axes = plt.subplots(  
2     2,  
3     2,  
4     figsize=(10, 8)  
5 )  
6  
7 axes[0, 0].plot(  
8     x,  
9     np.sin(x)  
10 )  
11  
12 axes[0, 0].set_title(  
13     "sin(x)"  
14 )  
15  
16 axes[0, 1].plot(  
17     x,  
18     np.cos(x)  
19 )  
20  
21 axes[0, 1].set_title(  
22     "cos(x)"  
23 )  
24  
25 axes[1, 0].plot(  
26     x,  
27     np.sin(2 * x)  
28 )  
29  
30 axes[1, 0].set_title(  
31     "sin(2x)"  
32 )  
33  
34 axes[1, 1].plot(  
35     x,  
36     np.cos(2 * x)  
37 )
```

```

38
39 axes[1, 1].set_title(
40     "cos(2x)"
41 )
42
43 fig.tight_layout()
44
45 plt.show()

```



In this case, axes behaves like a two-dimensional array of Axes objects.

This creates a useful connection with the array concepts considered previously:

```

1 axes[i, j]

```

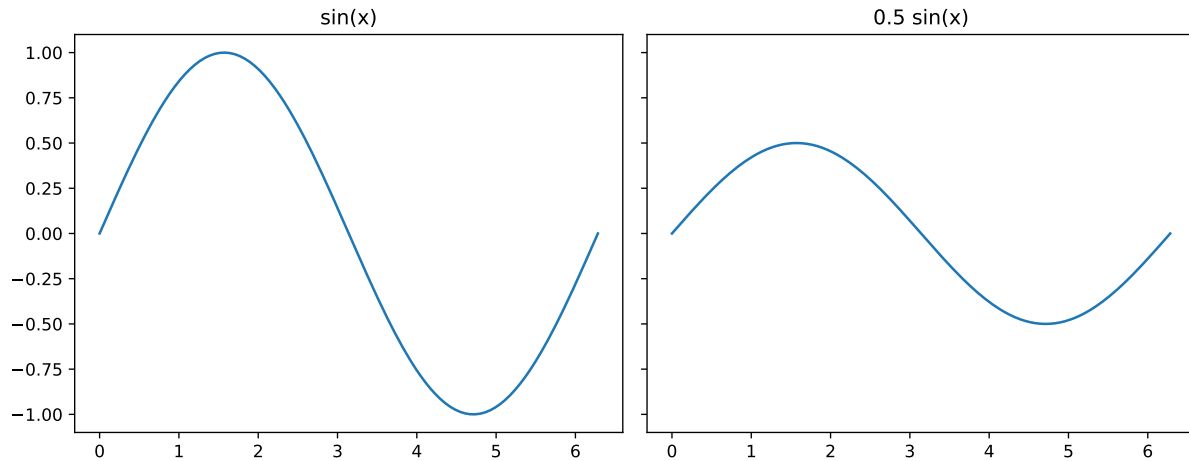
selects the plot at row (i) and column (j).

## 11.2 Shared Axes

When plots represent comparable quantities, shared scales can improve interpretation.

For example:

```
1 fig, axes = plt.subplots(  
2     1,  
3     2,  
4     figsize=(10, 4),  
5     sharey=True  
6 )  
7  
8 axes[0].plot(  
9     x,  
10    np.sin(x)  
11 )  
12  
13 axes[0].set_title(  
14     "sin(x)"  
15 )  
16  
17 axes[1].plot(  
18     x,  
19     0.5 * np.sin(x)  
20 )  
21  
22 axes[1].set_title(  
23     "0.5 sin(x)"  
24 )  
25  
26 fig.tight_layout()  
27  
28 plt.show()
```



Using the same vertical scale makes the difference in amplitude directly comparable.

If each subplot automatically used a very different scale, the visual comparison could become misleading.

### 11.3 Subplots Versus Overlaid Series

Several series can either be:

1. placed in the same Axes; or
2. separated into different Axes.

Overlaying series is useful when direct comparison on a common coordinate system is important.

Separate subplots are often preferable when:

- many series would overlap;
- scales differ substantially;
- each series contains substantial detail;
- the combined plot would require an excessively complex legend.

The choice is therefore an information-design decision rather than simply a formatting preference.

## 12 Visual Encoding and Scales

Every visualization maps data values to graphical properties.

A well-designed plot makes those mappings easy to interpret.

Important graphical properties include:

- position;
- length;
- colour;
- size;
- shape;
- line style;
- transparency.

Not all encodings communicate quantitative information equally well.

## 12.1 Position

Position along a common scale is one of the most precise visual encodings.

For example, differences between points in a scatter plot can often be compared accurately because their coordinates share the same axes.

This is one reason position is central to statistical graphics.

## 12.2 Length

Bar charts encode magnitude through length.

Comparisons are usually straightforward when bars share the same baseline.

As discussed previously, truncating the baseline can distort the perceived magnitude of differences.

## 12.3 Colour

Colour can represent either categorical or numerical information.

For categories, distinct colours may distinguish groups.

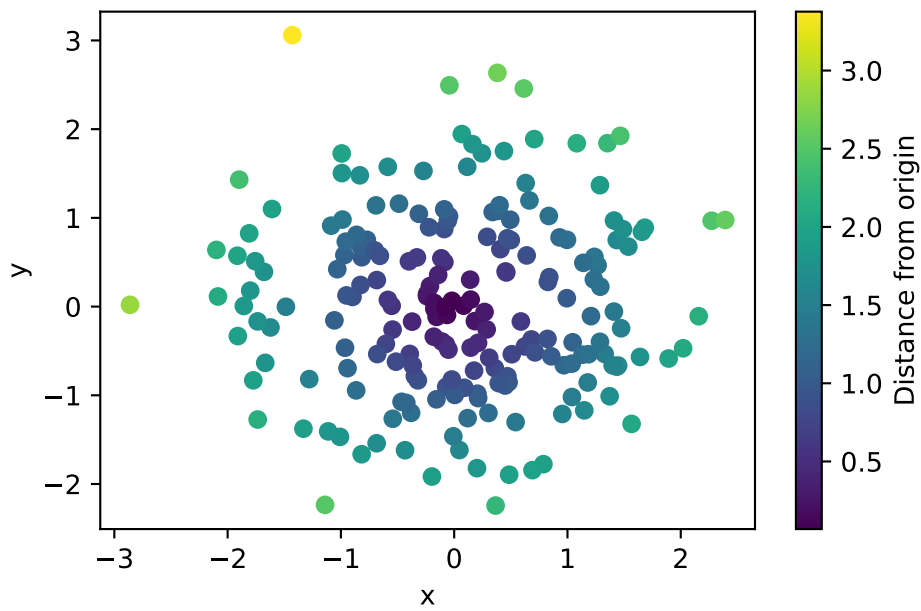
For continuous numerical values, a colour map can represent magnitude:

```
1 x = rng.normal(size=200)
2 y = rng.normal(size=200)
3
4 value = np.sqrt(
5     x ** 2 + y ** 2
```

```

6 )
7
8 fig, ax = plt.subplots()
9
10 scatter = ax.scatter(
11     x,
12     y,
13     c=value,
14     cmap="viridis"
15 )
16
17 fig.colorbar(
18     scatter,
19     ax=ax,
20     label="Distance from origin"
21 )
22
23 ax.set(
24     xlabel="x",
25     ylabel="y"
26 )
27
28 plt.show()

```



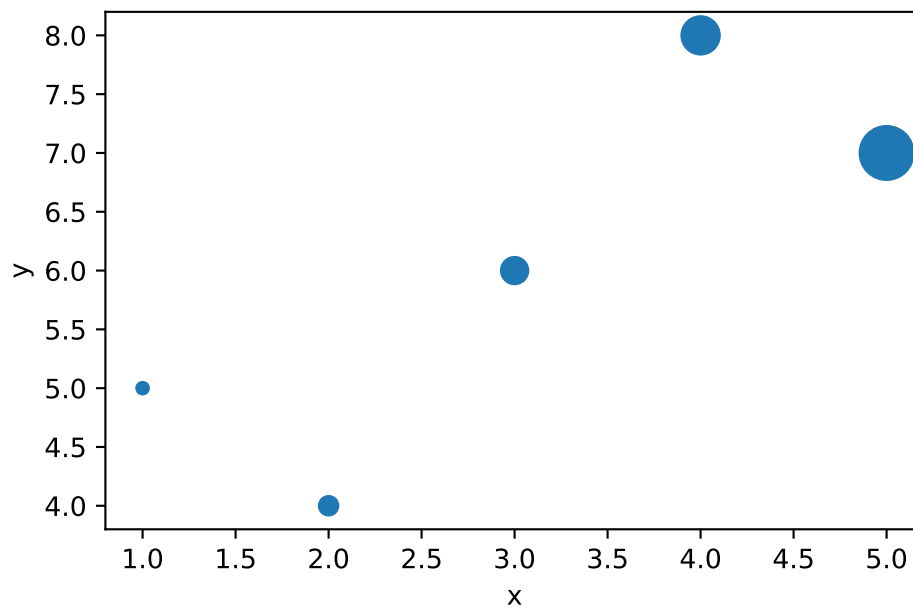


The colour scale should have a clear interpretation and, when numerical, should generally be accompanied by a colour bar.

## 12.4 Size

Marker size can encode a numerical quantity:

```
1 x = np.array([1, 2, 3, 4, 5])
2 y = np.array([5, 4, 6, 8, 7])
3
4 size = np.array([
5     20,
6     50,
7     100,
8     200,
9     400
10 ])
11
12 fig, ax = plt.subplots()
13
14 ax.scatter(
15     x,
16     y,
17     s=size
18 )
19
20 ax.set(
21     xlabel="x",
22     ylabel="y"
23 )
24
25 plt.show()
```



However, size comparisons are less precise than position comparisons.

For this reason, marker size is often most useful for communicating approximate magnitude rather than supporting precise numerical comparison.

## 12.5 Transparency

Transparency is controlled using alpha.

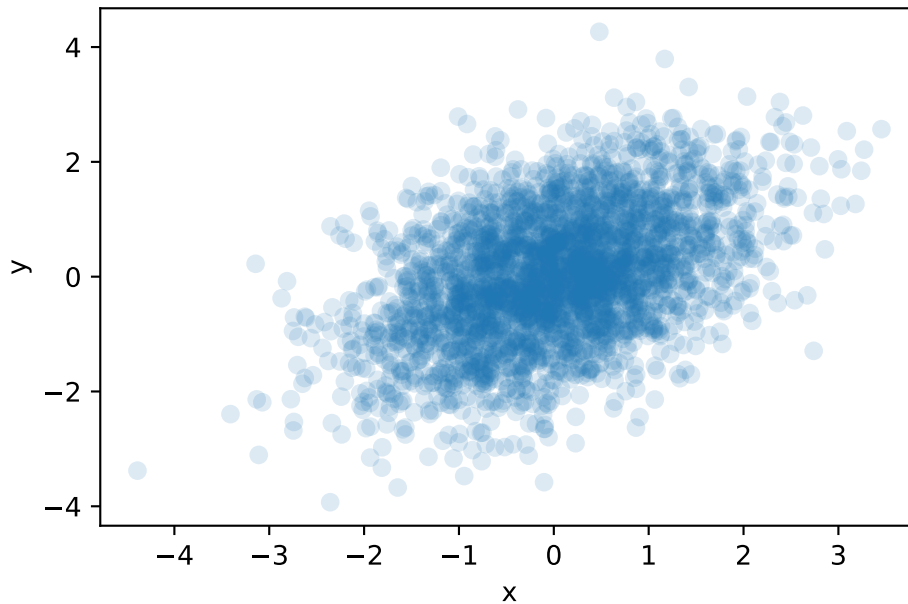
For example:

```
1 x = rng.normal(size=3000)
2 y = (
3     0.5 * x
4     + rng.normal(size=3000)
5 )
6
7 fig, ax = plt.subplots()
8
9 ax.scatter(
10     x,
11     y,
12     alpha=0.15
13 )
14
```

```

15 ax.set(
16     xlabel="x",
17     ylabel="y"
18 )
19
20 plt.show()

```



When observations overlap, transparency allows regions of high concentration to become visually apparent.

This is particularly useful with large datasets.

## 13 Axis Scales

The numerical scale of an axis determines how distances in data space are mapped to distances in the graphic.

The default is usually a **linear scale**.

On a linear scale, equal numerical differences correspond to equal graphical distances.

For example:

$$10 - 1 = 9$$

and:

$$100 - 91 = 9$$

produce the same difference in position.

### 13.1 Logarithmic Scales

When values span several orders of magnitude, a logarithmic scale may be more informative.

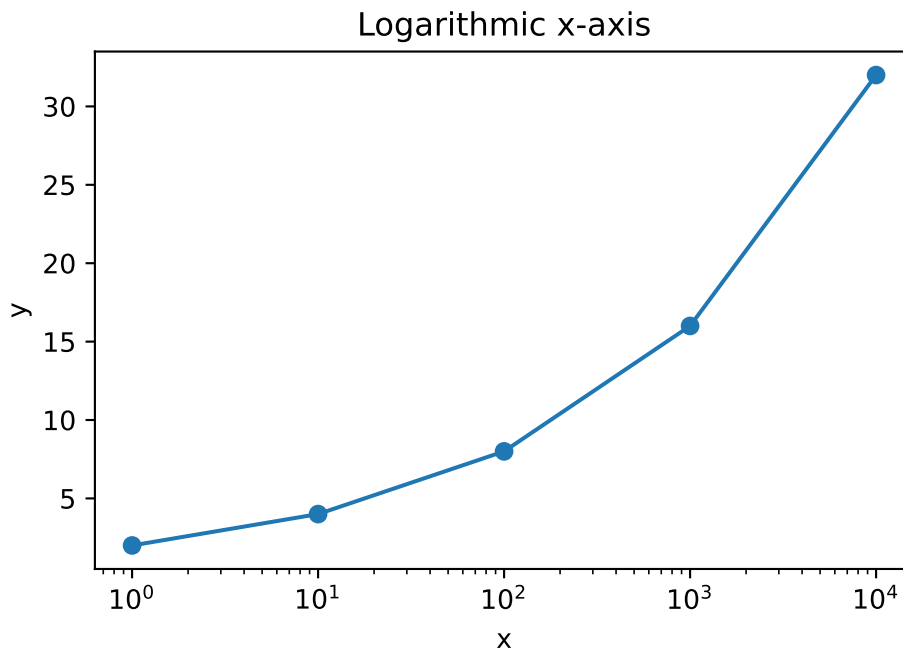
Consider:

```
1 x = np.array([
2     1,
3     10,
4     100,
5     1000,
6     10000
7 ])
8
9 y = np.array([
10    2,
11    4,
12    8,
13    16,
14    32
15 ])
16
17 fig, ax = plt.subplots()
18
19 ax.plot(
20     x,
21     y,
22     marker="o"
23 )
24
25 ax.set_xscale("log")
26
27 ax.set(
28     title="Logarithmic x-axis",
```

```

29 xlabel="x",
30 ylabel="y"
31 )
32
33 plt.show()

```



On a base-10 logarithmic scale, equal ratios rather than equal differences occupy equal distances:

$$1 \rightarrow 10 \rightarrow 100 \rightarrow 1000.$$

Each step represents multiplication by 10.

Logarithmic scales can be useful for quantities such as:

- population;
- income;
- probability ratios;
- model parameters spanning several orders of magnitude;
- training losses;
- scientific measurements.

However, a logarithmic scale changes the visual interpretation of distance and should therefore be communicated clearly.

## 14 Time-Series Visualization

Time introduces a natural ordering to observations.

For this reason, line plots are commonly used for time-series data.

Python provides several representations for dates and times. The standard library includes the `datetime` module.

For example:

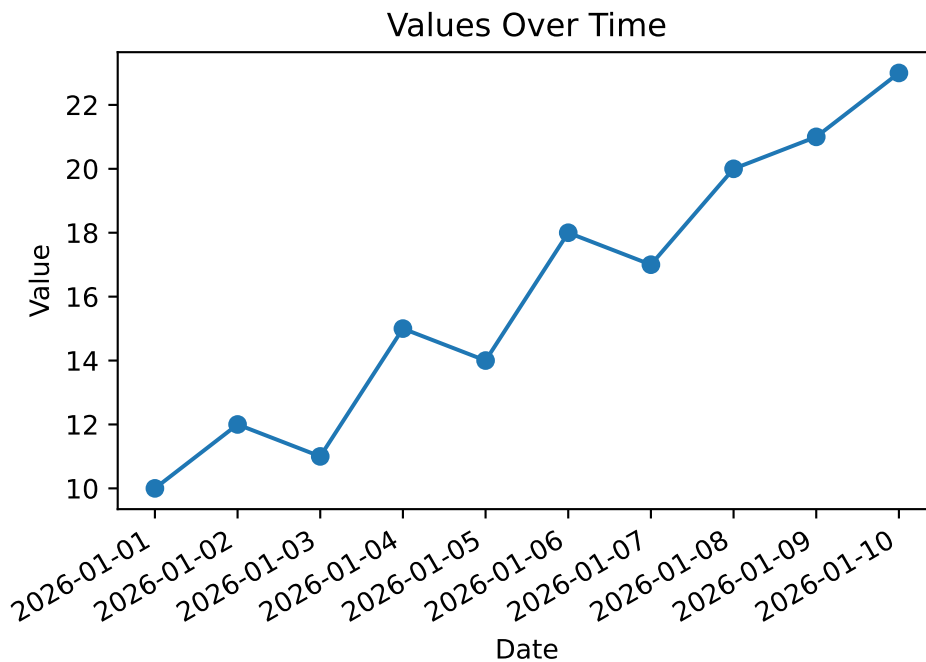
```
1 from datetime import datetime
2
3 date = datetime(
4     2026,
5     1,
6     1
7 )
8
9 date
```

```
datetime.datetime(2026, 1, 1, 0, 0)
```

A sequence of dates can be created and plotted directly with Matplotlib.

```
1 from datetime import timedelta
2
3 start = datetime(
4     2026,
5     1,
6     1
7 )
8
9 dates = [
10     start + timedelta(days=i)
11     for i in range(10)
12 ]
13
14 values = np.array([
15     10,
16     12,
17     11,
```

```
18     15,  
19     14,  
20     18,  
21     17,  
22     20,  
23     21,  
24     23  
25 ]) )  
26  
27 fig, ax = plt.subplots()  
28  
29 ax.plot(  
30     dates,  
31     values,  
32     marker="o"  
33 )  
34  
35 ax.set(  
36     title="Values Over Time",  
37     xlabel="Date",  
38     ylabel="Value"  
39 )  
40  
41 fig.autofmt_xdate()  
42  
43 plt.show()
```



The horizontal axis now represents chronological time.

## 14.1 Time as an Ordered Dimension

In a time series:

$$y_t$$

represents the value of a variable observed at time (t).

The ordering is intrinsic:

$$t_1 < t_2 < \dots < t_n.$$

Connecting observations with lines is therefore meaningful because consecutive observations correspond to consecutive positions in time.

A time-series plot can reveal:

- trends;
- seasonal behaviour;
- cycles;
- sudden changes;

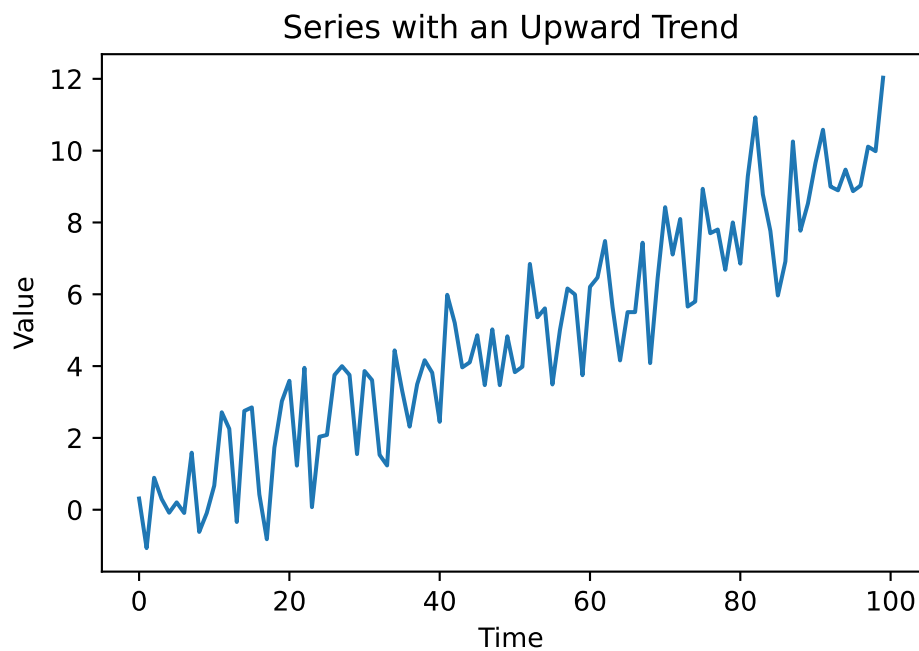


- periods of high or low variability;
- anomalous observations.

## 14.2 Trend

Consider:

```
1  n = 100
2
3  time = np.arange(n)
4
5  trend = (
6      0.1 * time
7      + rng.normal(
8          scale=1,
9          size=n
10     )
11 )
12
13 fig, ax = plt.subplots()
14
15 ax.plot(
16     time,
17     trend
18 )
19
20 ax.set(
21     title="Series with an Upward Trend",
22     xlabel="Time",
23     ylabel="Value"
24 )
25
26 plt.show()
```



The upward tendency becomes easier to identify graphically than from the raw numerical values alone.

### 14.3 Seasonality

A repeated temporal pattern can be represented as:

```

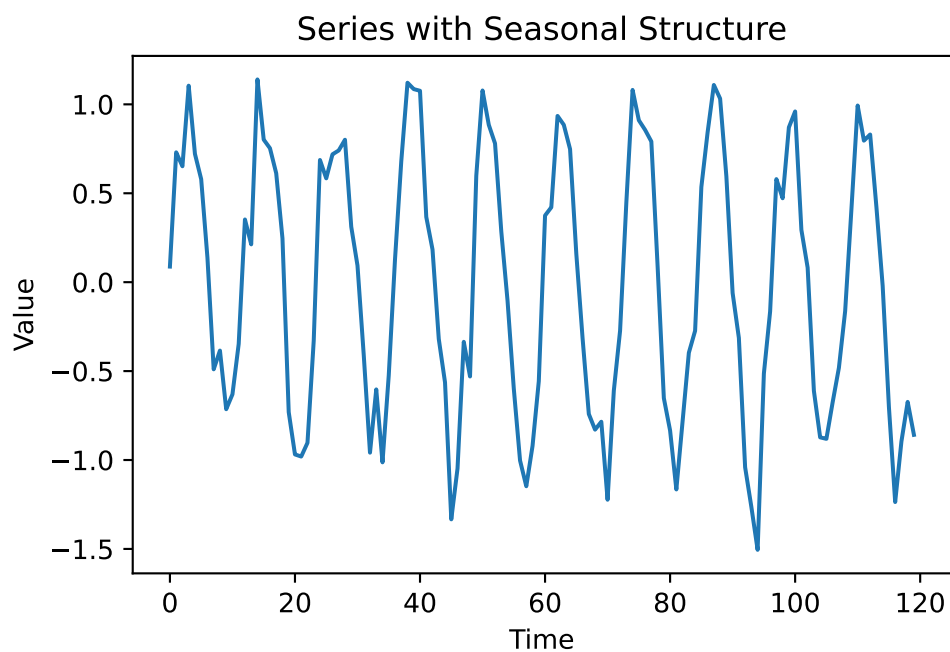
1  n = 120
2
3  time = np.arange(n)
4
5  seasonal = (
6      np.sin(
7          2 * np.pi * time / 12
8      )
9      + rng.normal(
10         scale=0.2,
11         size=n
12     )
13 )
14
15 fig, ax = plt.subplots()
16

```

```

17 ax.plot(
18     time,
19     seasonal
20 )
21
22 ax.set(
23     title="Series with Seasonal Structure",
24     xlabel="Time",
25     ylabel="Value"
26 )
27
28 plt.show()

```



The periodic structure is visually apparent.

Graphical inspection is therefore often one of the first steps in time-series analysis.

## 14.4 Multiple Time Series

Related time series can be compared on the same Axes:

```

1 time = np.arange(50)
2
3 series_a = (
4     0.1 * time
5     + rng.normal(
6         scale=0.5,
7         size=50
8     )
9 )
10
11 series_b = (
12     0.15 * time
13     + rng.normal(
14         scale=0.5,
15         size=50
16     )
17 )
18
19 fig, ax = plt.subplots()
20
21 ax.plot(
22     time,
23     series_a,
24     label="Series A "
25 )
26
27 ax.plot(
28     time,
29     series_b,
30     label="Series B"
31 )
32
33 ax.set(
34     title="Comparison of Two Time Series",
35     xlabel="Time",
36     ylabel="Value"
37 )
38
39 ax.legend()
40

```



A shared time axis makes differences in level, trend, and variation easier to compare.

However, if the series use fundamentally different units or scales, placing them on the same axis may be difficult to interpret.

## 14.5 Time-Series Plots and Causality

Temporal ordering provides information about **when** events occur, but temporal association alone does not establish causality.

If one series changes before another, this may motivate further analysis, but it does not by itself demonstrate that the first variable caused the second.

As with scatter plots:

visual association  $\nRightarrow$  causal relationship.

Graphical representation is primarily a tool for identifying structure and generating or evaluating analytical questions. Causal interpretation requires additional assumptions and methodology.

## 15 Seaborn and Statistical Visualization

**Seaborn** is a statistical visualization library built on top of Matplotlib.

It provides a higher-level interface for constructing common statistical graphics and is particularly useful when data contain combinations of:

- numerical variables;
- categorical variables;
- groups;
- distributions;
- statistical relationships.

The conventional import is:

```
1 import seaborn as sns
```

Seaborn does not replace Matplotlib. Rather, it provides additional abstractions while continuing to use Matplotlib's figure and Axes system underneath.

This means that a Seaborn plot can still be customized through Matplotlib.

For example, consider a small dataset represented using a pandas DataFrame:

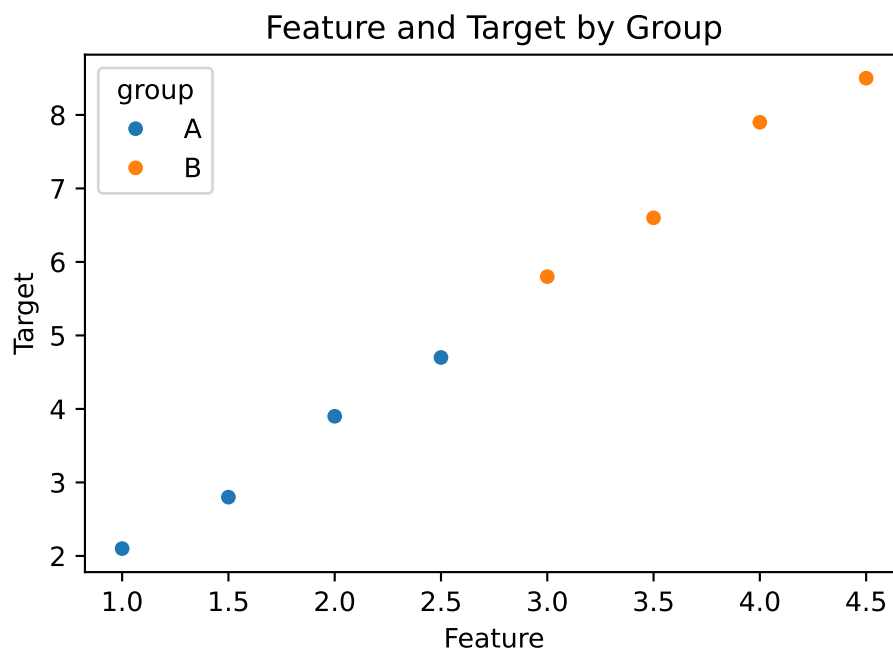
```
1 import pandas as pd
2
3 data = pd.DataFrame({
4     "feature": [
5         1.0, 1.5, 2.0, 2.5,
6         3.0, 3.5, 4.0, 4.5
7     ],
8     "target": [
9         2.1, 2.8, 3.9, 4.7,
10        5.8, 6.6, 7.9, 8.5
11    ],
12    "group": [
13        "A", "A", "A", "A",
14        "B", "B", "B", "B"
15    ]
16 })
```

A scatter plot can be created using:

```

1 fig, ax = plt.subplots()
2
3 sns.scatterplot(
4     data=data,
5     x="feature",
6     y="target",
7     hue="group",
8     ax=ax
9 )
10
11 ax.set(
12     title="Feature and Target by Group",
13     xlabel="Feature",
14     ylabel="Target"
15 )
16
17 plt.show()

```



The important difference is that the graphical mapping is expressed directly using variable names:

```

1 x="feature"
2 y="target"

```

```
3 hue="group"
```

This makes Seaborn particularly convenient when working with tabular datasets.

## 15.1 Semantic Mappings

Seaborn uses graphical properties to represent additional variables.

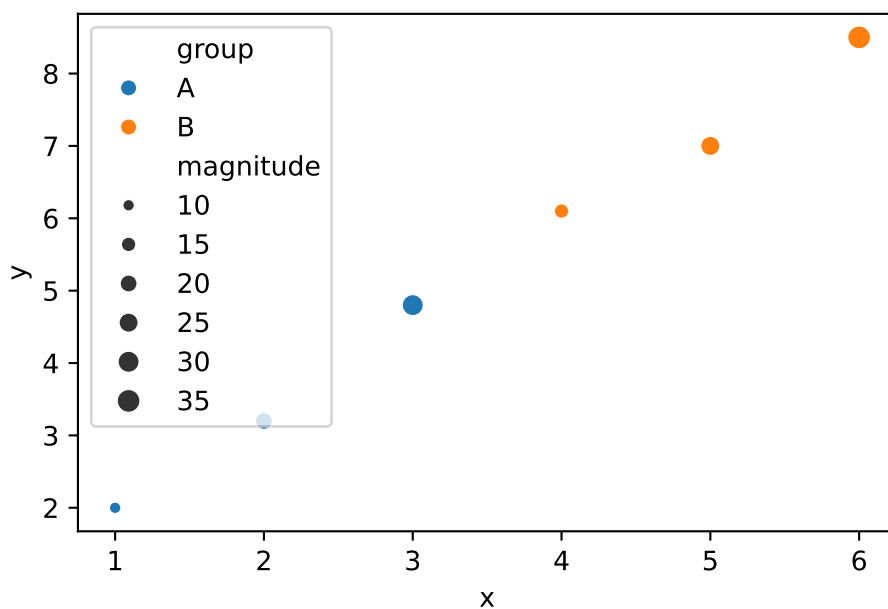
Common semantic mappings include:

- x: horizontal position;
- y: vertical position;
- hue: colour;
- size: marker size;
- style: marker or line style.

For example:

```
1 data = pd.DataFrame({
2     "x": [1, 2, 3, 4, 5, 6],
3     "y": [2.0, 3.2, 4.8, 6.1, 7.0, 8.5],
4     "group": ["A", "A", "A", "B", "B", "B"],
5     "magnitude": [10, 20, 30, 15, 25, 35]
6 })
7
8 fig, ax = plt.subplots()
9
10 sns.scatterplot(
11     data=data,
12     x="x",
13     y="y",
14     hue="group",
15     size="magnitude",
16     ax=ax
17 )
18
19 plt.show()
```





Here, several variables are encoded simultaneously:

$x \longrightarrow$  horizontal position,

$y \longrightarrow$  vertical position,

group  $\longrightarrow$  colour,

and:

magnitude  $\longrightarrow$  marker size.

The ability to specify these mappings declaratively is one of Seaborn's main advantages.

## 16 Statistical Relationships with Seaborn

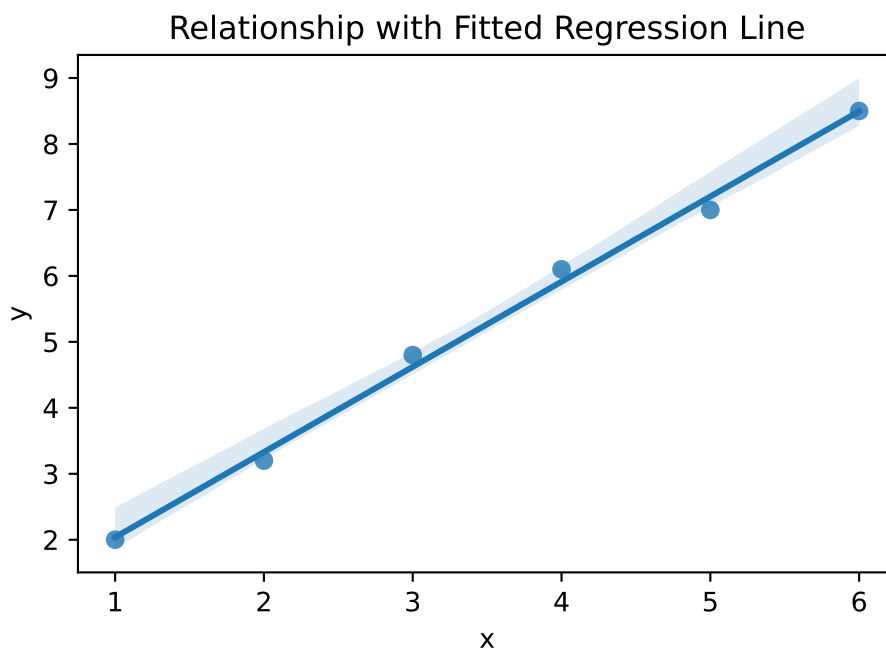
Seaborn provides higher-level functions for investigating statistical relationships.

For example:

```

1 fig, ax = plt.subplots()
2
3 sns.regplot(
4     data=data,
5     x="x",
6     y="y",
7     ax=ax
8 )
9
10 ax.set(
11     title="Relationship with Fitted Regression Line"
12 )
13
14 plt.show()

```



A regression plot combines:

- the observed data;
- a fitted relationship;
- an uncertainty representation around that relationship.

This can be useful during exploratory analysis, but the fitted line should still be interpreted as a statistical model rather than simply a graphical decoration.

A visually strong linear pattern does not by itself establish:

- causality;
- model adequacy;
- independence of errors;
- constant variance;
- absence of influential observations.

Visualization helps investigate a model; it does not replace statistical reasoning.

## 17 Visualizing Distributions with Seaborn

Seaborn provides several functions for representing numerical distributions.

Consider:

```

1 rng = np.random.default_rng(42)
2
3 sample = pd.DataFrame({
4     "value": rng.normal(
5         loc=0,
6         scale=1,
7         size=1000
8     )
9 })

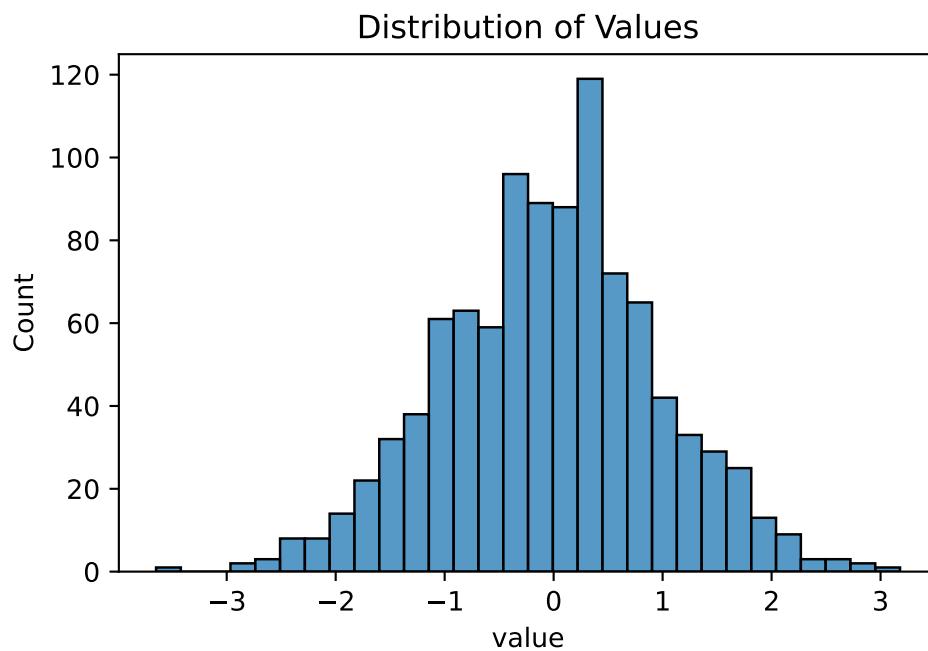
```

A histogram can be created with:

```

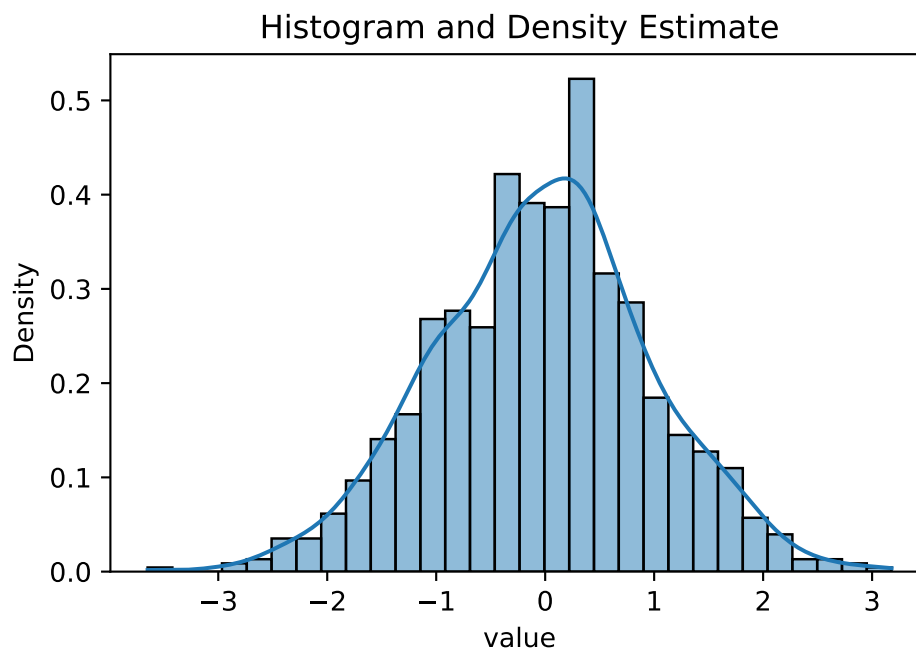
1 fig, ax = plt.subplots()
2
3 sns.histplot(
4     data=sample,
5     x="value",
6     bins=30,
7     ax=ax
8 )
9
10 ax.set(
11     title="Distribution of Values"
12 )
13
14 plt.show()

```



Seaborn can also add a kernel density estimate:

```
1 fig, ax = plt.subplots()
2
3 sns.histplot(
4     data=sample,
5     x="value",
6     bins=30,
7     stat="density",
8     kde=True,
9     ax=ax
10 )
11
12 ax.set(
13     title="Histogram and Density Estimate"
14 )
15
16 plt.show()
```



The histogram represents a binned empirical distribution, whereas the density curve provides a smoothed representation.

The two should not be confused: the density estimate introduces an additional modelling choice through the degree of smoothing.

## 17.1 Box Plots by Group

Categorical comparisons can also be represented naturally:

```

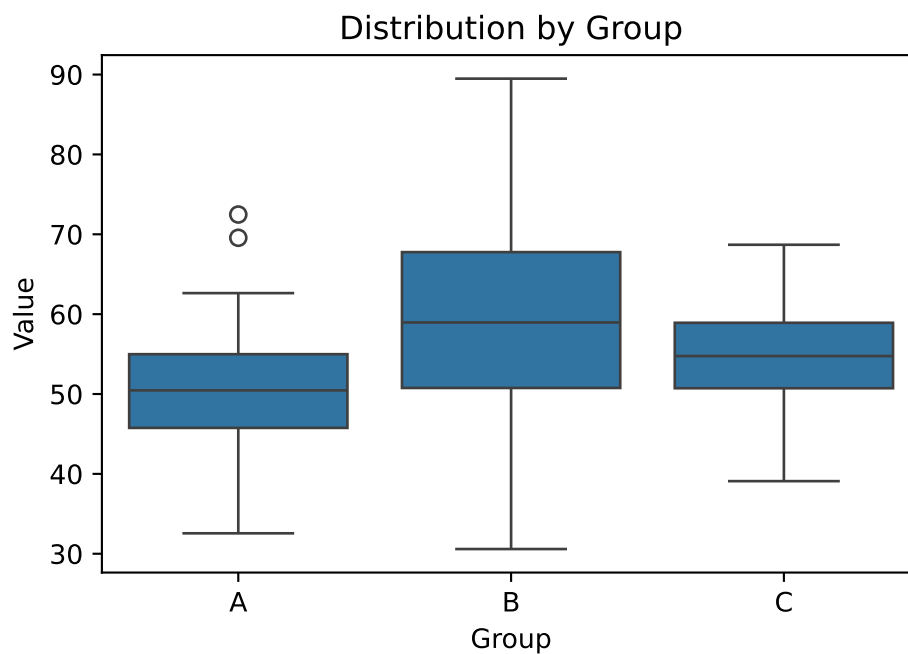
1 grouped_data = pd.DataFrame({
2     "group": (
3         ["A"] * 100
4         + ["B"] * 100
5         + ["C"] * 100
6     ),
7     "value": np.concatenate([
8         rng.normal(50, 8, 100),
9         rng.normal(60, 12, 100),
10        rng.normal(55, 6, 100)
11    ])
12 })

```

```

1 fig, ax = plt.subplots()
2
3 sns.boxplot(
4     data=grouped_data,
5     x="group",
6     y="value",
7     ax=ax
8 )
9
10 ax.set(
11     title="Distribution by Group",
12     xlabel="Group",
13     ylabel="Value"
14 )
15
16 plt.show()

```



Seaborn automatically maps the categorical variable to separate distributions.

This is an example of why high-level statistical visualization libraries can be useful: the code describes the analytical structure rather than manually constructing every graphical component.

## 18 Faceting

Sometimes the clearest way to represent an additional categorical variable is not through colour or marker style, but by dividing the visualization into several panels.

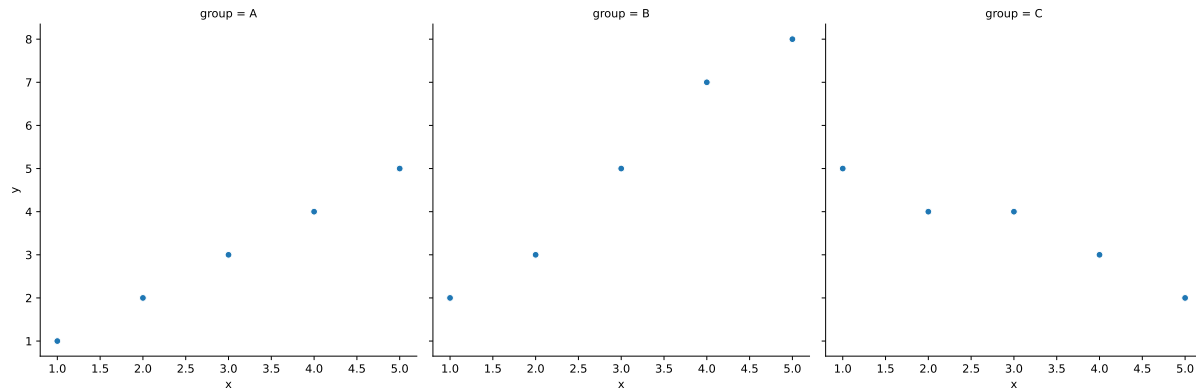
This approach is called **faceting**.

Consider:

```
1 data = pd.DataFrame({
2     "x": np.tile(
3         np.arange(1, 6),
4         3
5     ),
6     "y": np.concatenate([
7         np.array([1, 2, 3, 4, 5]),
8         np.array([2, 3, 5, 7, 8]),
9         np.array([5, 4, 4, 3, 2])
10    ]),
11     "group": (
12         ["A"] * 5
13         + ["B"] * 5
14         + ["C"] * 5
15     )
16 })
```

A faceted representation can be created with:

```
1 g = sns.relplot(
2     data=data,
3     x="x",
4     y="y",
5     col="group",
6     kind="scatter"
7 )
8
9 g.set_axis_labels(
10     "x",
11     "y"
12 )
13
14 plt.show()
```



Each panel represents one value of the categorical variable.

Faceting is useful when placing all groups in the same Axes would produce:

- excessive overlap;
- a complicated legend;
- difficult comparisons;
- visually crowded plots.

It represents another general visualization principle:

When additional graphical encodings create too much complexity, separating the data spatially can improve interpretation.

## 19 Choosing an Appropriate Visualization

There is no universally best plot.

The appropriate representation depends on:

1. the analytical question;
2. the types of variables involved;
3. the structure of the data;
4. the comparison the viewer needs to make.

A useful starting point is to identify the variables involved.



## 19.1 One Numerical Variable

If the objective is to understand the distribution of one numerical variable, useful representations include:

- histogram;
- density plot;
- box plot.

A histogram or density plot emphasizes the **shape of the distribution**.

A box plot provides a more compressed summary emphasizing:

- median;
- quartiles;
- spread;
- potential extreme observations.

## 19.2 One Categorical and One Numerical Variable

If the objective is to compare a numerical quantity across categories, possible representations include:

- bar plot for an aggregate;
- box plot for distributions;
- grouped distribution plots.

For example, a bar plot of mean income by region does not show the distribution of income within each region.

A box plot provides different information.

The graphical form should therefore reflect whether the analytical target is an **aggregate** or a **distribution**.

## 19.3 Two Numerical Variables

For two numerical variables, a scatter plot is usually a natural starting point.

It allows inspection of:

- direction of association;
- strength of association;
- nonlinearity;

- clusters;
- outliers;
- heteroscedasticity.

A fitted line may be added when the analytical question concerns a modelled relationship.

## 19.4 Numerical Variable Over an Ordered Dimension

For observations indexed by:

- time;
- iteration;
- distance;
- model complexity;
- another ordered variable,

a line plot is often appropriate.

The connecting line represents the meaningful progression between successive observations.

## 19.5 Several Numerical Variables

When several numerical variables must be investigated simultaneously, possible strategies include:

- multiple scatter plots;
- correlation matrices;
- faceting;
- carefully selected colour or size mappings.

The goal is not to place every available variable into one visualization. Increasing the number of encodings can eventually reduce rather than increase understanding.

## 20 Pie Charts

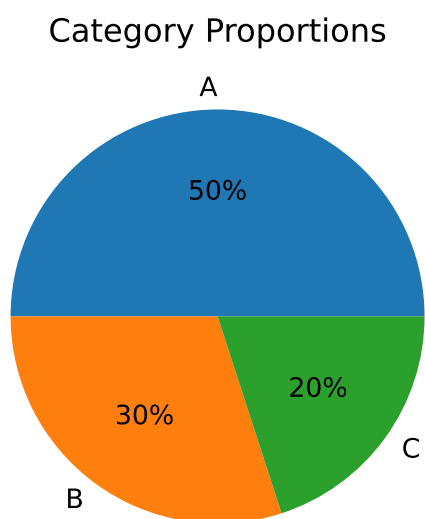
Matplotlib can create pie charts using `pie()`.

For example:

```

1 categories = [
2     "A",
3     "B",
4     "C"
5 ]
6
7 values = [
8     50,
9     30,
10    20
11 ]
12
13 fig, ax = plt.subplots()
14
15 ax.pie(
16     values,
17     labels=categories,
18     autopct="%1.0f%%"
19 )
20
21 ax.set_title(
22     "Category Proportions"
23 )
24
25 plt.show()

```



A pie chart maps proportions to **angles and areas**.

Although pie charts are widely used, they are often less effective than bar plots when accurate comparison is important.

Humans generally compare positions and lengths more accurately than angles and areas.

For example, distinguishing:

24%

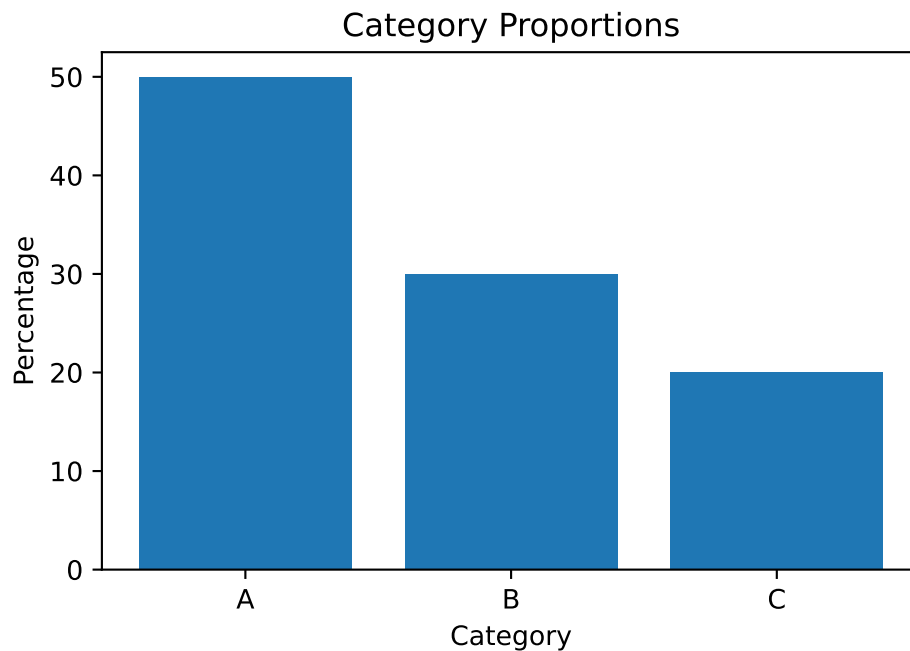
from:

27%

may be difficult in a pie chart, whereas the corresponding difference in bar lengths is easier to perceive.

A bar plot using the same data would be:

```
1 fig, ax = plt.subplots()
2
3 ax.bar(
4     categories,
5     values
6 )
7
8 ax.set(
9     title="Category Proportions",
10    xlabel="Category",
11    ylabel="Percentage"
12 )
13
14 plt.show()
```



Pie charts may still be understandable when:

- the number of categories is small;
- differences are substantial;
- the purpose is to communicate approximate part-to-whole composition.

They become increasingly difficult to interpret when:

- many categories are present;
- values are similar;
- exact comparison matters.

The important principle is not that pie charts are universally incorrect, but that graphical form should be chosen according to the comparison the viewer must perform.

## 21 Common Visualization Problems

A graph can be technically correct while still communicating the data poorly.

Visualization therefore requires attention not only to syntax but also to interpretation.

## 21.1 Truncated Axes

As discussed for bar plots, a truncated baseline can exaggerate differences when bar length represents magnitude.

Suppose two values are:

$$98 \quad \text{and} \quad 100.$$

Displaying only the range:

$$97 \leq y \leq 101$$

can make the difference appear visually dramatic.

Whether truncation is misleading depends partly on the graphical representation and analytical purpose.

For a line plot examining small fluctuations, a restricted range may be useful.

For bars whose lengths represent absolute magnitude, a zero baseline is usually more appropriate.

## 21.2 Excessive Visual Complexity

A visualization can include:

- multiple colours;
- several marker types;
- different sizes;
- several line styles;
- annotations;
- legends;
- multiple axes.

Each element may be individually meaningful while the complete plot becomes difficult to decode.

A good visualization generally uses only the encodings required to communicate the analytical question.

### 21.3 Overplotting

When many observations overlap, a scatter plot may conceal the actual density of the data.

Possible responses include:

- transparency;
- smaller markers;
- sampling;
- aggregation;
- density-based representations.

The correct response depends on what information should remain visible.

### 21.4 Inappropriate Aggregation

Summary graphics can conceal important distributions.

Suppose two groups have the same mean:

$$\bar{x}_A = \bar{x}_B.$$

This does not imply that the groups have the same:

- variance;
- skewness;
- range;
- modality;
- outliers.

A bar chart displaying only group means could therefore hide substantial differences.

Whenever data are aggregated for visualization, it is useful to ask:

**What information disappeared during aggregation?**

### 21.5 Dual Axes

Plots containing two vertical axes can sometimes be useful when series have different units, but they also create substantial interpretive risk.

Different axis ranges can visually amplify or suppress relationships between the series.

Whenever possible, alternatives such as:

- separate panels;
- standardized values;
- indexed series

may provide clearer comparisons.

## 21.6 Uncertainty

An estimated value is not always known exactly.

For example, a model performance estimate or group mean may have sampling uncertainty.

When uncertainty is relevant, visualizations may include:

- error bars;
- confidence intervals;
- prediction intervals;
- distributions of estimates.

Displaying a point estimate without its uncertainty can communicate more precision than the analysis supports.

The appropriate uncertainty representation depends on the statistical quantity being shown.

## 21.7 Graphical Association and Causal Interpretation

Patterns in data can suggest relationships, but visualization alone does not establish causality.

This principle applies to:

- scatter plots;
- regression plots;
- time-series plots;
- grouped comparisons.

Potential explanations for an observed relationship include:

- causal effects;
- confounding;
- selection mechanisms;
- common trends;
- measurement processes;
- chance.



A visualization is evidence about the **observed structure of the data**, not automatically about the mechanism that generated it.

## 22 Network Visualization

Most visualizations considered so far represent observations through positions on numerical or categorical axes.

Some datasets instead describe **relationships between entities**.

These structures can be represented mathematically as graphs.

A graph is commonly written as:

$$G = (V, E),$$

where:

- $V$  is a set of **vertices**, or nodes;
- $E$  is a set of **edges** connecting pairs of nodes.

Python provides the **NetworkX** library for constructing, analysing, and visualizing graph structures.

The conventional import is:

```
1 import networkx as nx
```

### 22.1 Creating a Graph

An undirected graph can be created with:

```
1 G = nx.Graph()
```

Nodes can then be added:

```
1 G.add_nodes_from([
2     "A",
3     "B",
4     "C",
5     "D",
6     "E"
7 ])
```

and edges:

```
1 G.add_edges_from([
2     ("A", "B"),
3     ("A", "C"),
4     ("B", "D"),
5     ("C", "D"),
6     ("D", "E")
7 ])
```

The graph now represents binary relationships among five entities.

Its nodes can be inspected using:

```
1 list(G.nodes)
```

```
['A', 'B', 'C', 'D', 'E']
```

and its edges using:

```
1 list(G.edges)
```

```
[('A', 'B'), ('A', 'C'), ('B', 'D'), ('C', 'D'), ('D', 'E')]
```

## 22.2 Neighbours and Degree

Two nodes are neighbours when an edge connects them.

For example:

```
1 list(G.neighbors("A"))
```

```
['B', 'C']
```

The **degree** of a node is the number of edges incident to it.

```
1 dict(G.degree)
```

```
{'A': 2, 'B': 2, 'C': 2, 'D': 3, 'E': 1}
```

For an undirected graph, the degree of vertex  $v$  can be written as:

$$d(v) = |\{u \in V : (u, v) \in E\}|.$$

The degree provides a simple measure of local connectivity.

More advanced network analysis can use substantially richer concepts, but degree is a useful starting point.

## 22.3 Weighted Edges

Edges may also contain additional information.

For example:

```
1 G_weighted = nx.Graph()
2
3 G_weighted.add_weighted_edges_from([
4     ("A", "B", 2.0),
5     ("A", "C", 1.5),
6     ("B", "D", 3.0),
7     ("C", "D", 2.5),
8     ("D", "E", 4.0)
9 ])
```

The edge weights might represent quantities such as:

- distance;
- cost;
- frequency;

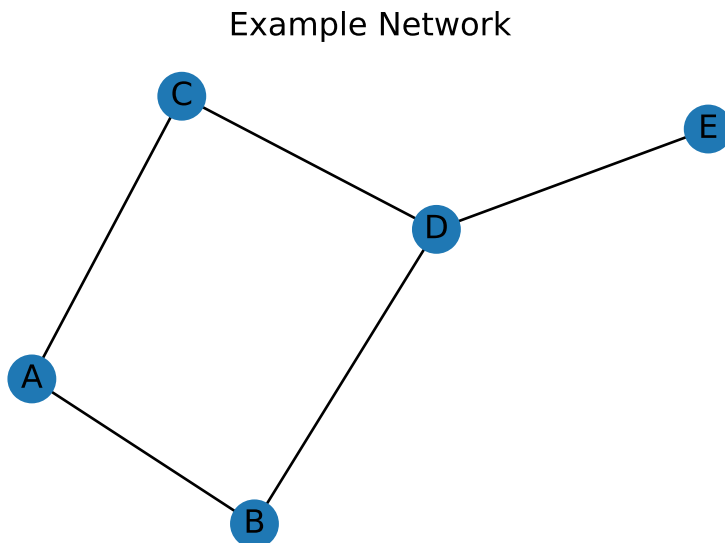
- strength of interaction;
- similarity.

The meaning of the weight depends entirely on the application.

## 22.4 Drawing a Graph

A simple network representation can be created with:

```
1 fig, ax = plt.subplots()
2
3 nx.draw(
4     G,
5     with_labels=True,
6     ax=ax
7 )
8
9 ax.set_title(
10     "Example Network"
11 )
12
13 plt.show()
```



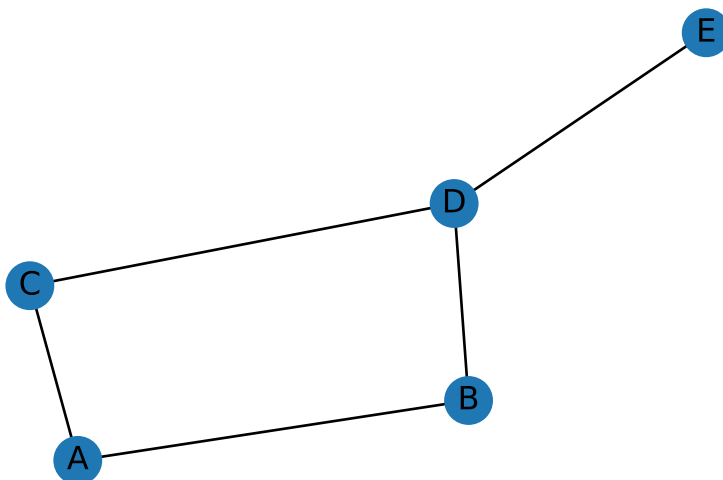
Unlike ordinary scatter plots, the node positions in many network visualizations are not direct numerical measurements.

Instead, a **layout algorithm** chooses positions to make the network structure easier to inspect.

For example:

```
1 position = nx.spring_layout(  
2     G,  
3     seed=42  
4 )  
5  
6 fig, ax = plt.subplots()  
7  
8 nx.draw(  
9     G,  
10    pos=position,  
11    with_labels=True,  
12    ax=ax  
13 )  
14  
15 ax.set_title(  
16     "Network Using a Spring Layout"  
17 )  
18  
19 plt.show()
```

Network Using a Spring Layout



The geometric distance between nodes in such a diagram should therefore not automatically be interpreted as an observed numerical distance.

The locations exist primarily to reveal network structure.

This distinguishes network graphics conceptually from plots whose axes encode measured quantities.

## 23 Visualization in Data Science and Machine Learning

Visualization plays several different roles across a data science workflow.

It is useful to distinguish those roles because different stages require different graphical questions.

### 23.1 Data Understanding

Before fitting a model, visualization can help identify:

- variable distributions;
- missing or unusual values;
- potential outliers;
- relationships between features;
- class imbalance;
- temporal structure;
- grouping effects.

Graphical exploration can therefore reveal modelling issues before a formal model is constructed.

### 23.2 Model Diagnostics

After fitting a model, graphical representations can help investigate whether its behaviour is consistent with assumptions.

For regression, examples include:

- residuals versus fitted values;
- residual distributions;
- observed versus predicted values.

Suppose:

```

1 targets = np.array([
2     2.1,
3     3.4,
4     4.8,
5     6.0,
6     7.2,
7     8.5
8 ])
9
10 predictions = np.array([
11     2.0,
12     3.6,
13     4.5,
14     6.2,
15     7.0,
16     8.3
17 ])
18
19 residuals = (
20     targets - predictions
21 )

```

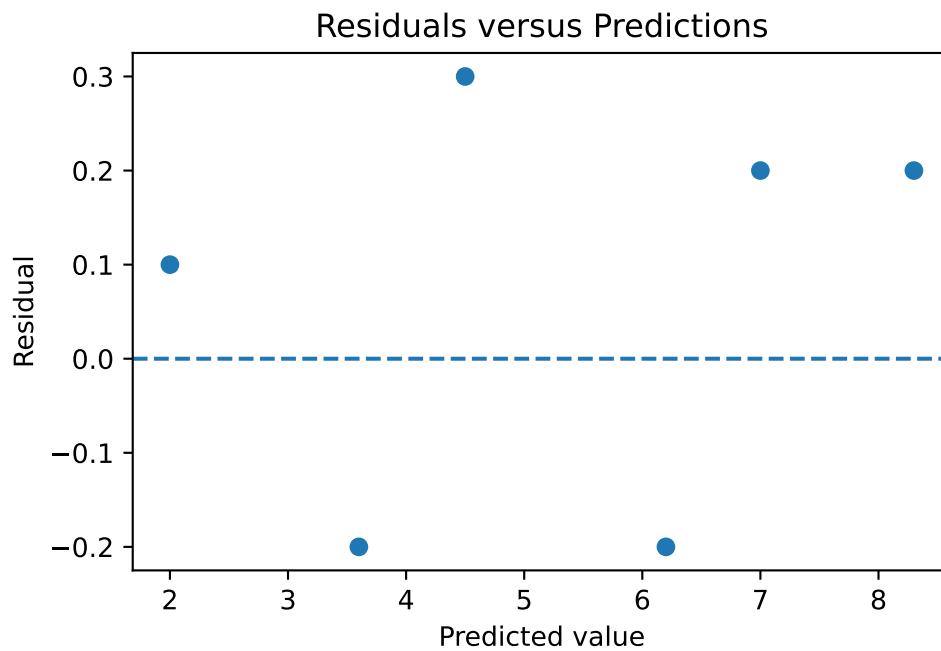
A residual plot can be created with:

```

1 fig, ax = plt.subplots()
2
3 ax.scatter(
4     predictions,
5     residuals
6 )
7
8 ax.axhline(
9     0,
10    linestyle="--"
11 )
12
13 ax.set(
14     title="Residuals versus Predictions",
15     xlabel="Predicted value",
16     ylabel="Residual"

```

```
17 )  
18  
19 plt.show()
```



The reference line at:

$$e = 0$$

provides a visual baseline.

Systematic structure in residuals can suggest that the model is failing to capture some aspect of the relationship.

However, graphical diagnostics must still be interpreted together with statistical and domain knowledge.

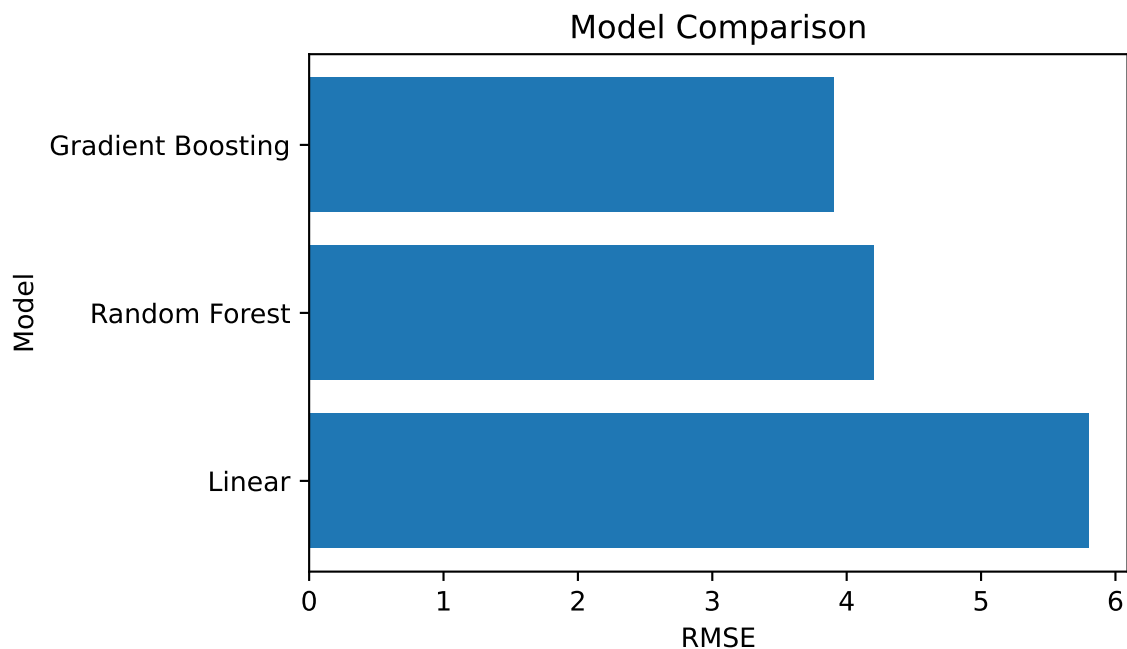
### 23.3 Model Comparison

Model performance can also be represented graphically.

For example:



```
1 models = np.array([
2     "Linear",
3     "Random Forest",
4     "Gradient Boosting"
5 ])
6
7 rmse = np.array([
8     5.8,
9     4.2,
10    3.9
11 ])
12
13 fig, ax = plt.subplots()
14
15 ax.barh(
16     models,
17     rmse
18 )
19
20 ax.set(
21     title="Model Comparison",
22     xlabel="RMSE",
23     ylabel="Model"
24 )
25
26 plt.show()
```



Because lower RMSE represents better performance, the numerical meaning of the metric must be understood before interpreting the visualization.

The graphic communicates the comparison; it does not determine which metric is appropriate.

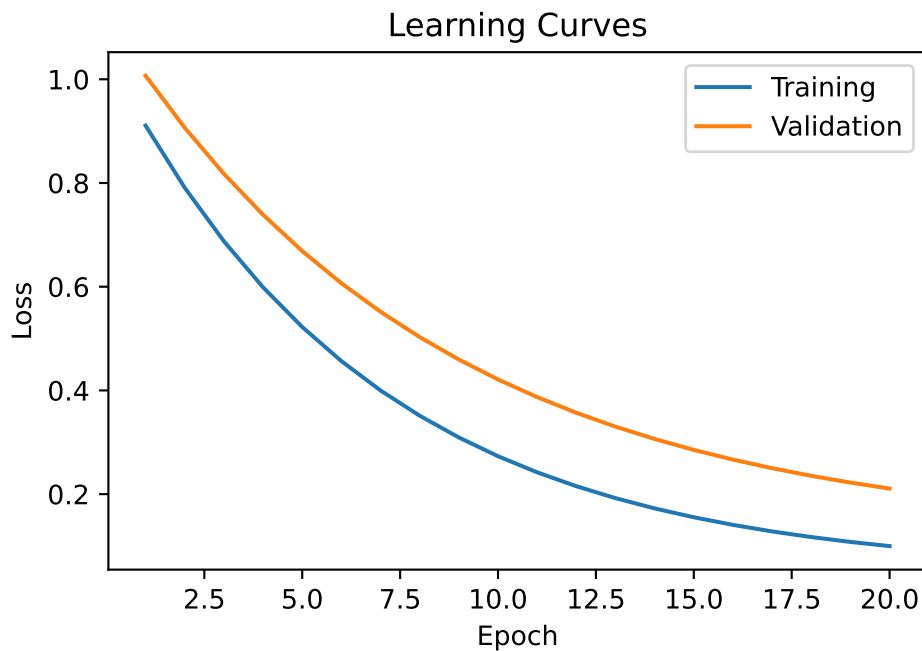
## 23.4 Learning Curves

Model behaviour across training can be represented using an ordered axis.

For example:

```
1 epochs = np.arange(  
2     1,  
3     21  
4 )  
5  
6 training_loss = np.exp(  
7     -0.15 * epochs  
8 ) + 0.05  
9  
10 validation_loss = (  
11     np.exp(  
12         -0.12 * epochs
```

```
13     )
14     + 0.12
15 )
16
17 fig, ax = plt.subplots()
18
19 ax.plot(
20     epochs,
21     training_loss,
22     label="Training"
23 )
24
25 ax.plot(
26     epochs,
27     validation_loss,
28     label="Validation"
29 )
30
31 ax.set(
32     title="Learning Curves",
33     xlabel="Epoch",
34     ylabel="Loss"
35 )
36
37 ax.legend()
38
39 plt.show()
```



Learning curves may help identify behaviour such as:

- continued optimization;
- convergence;
- divergence between training and validation performance;
- potential overfitting.

Again, the visualization becomes useful because the horizontal dimension has a meaningful ordering.

### 23.5 Visualization as Part of Analytical Reasoning

The principal purpose of visualization in Data Science and Machine Learning is not to make analytical outputs visually attractive.

It is to make particular aspects of the data or model easier to reason about.

When constructing or evaluating a graphic, useful questions include:

1. What analytical question is the plot intended to answer?
2. What does each axis represent?
3. What variable is encoded by each graphical property?
4. Are observations or aggregates being displayed?
5. Does the scale support a fair comparison?
6. Is important information being hidden through aggregation or overlap?

7. Is uncertainty relevant?
8. Does the visualization imply continuity, magnitude, or causality that the data do not support?
9. Would another graphical form make the intended comparison easier?
10. What conclusion is actually supported by the visualization?

Developing this habit is more transferable than memorizing plotting-library parameters.

## 24 Key Takeaways

Graphical representation is an analytical tool for making structure in data easier to perceive.

A visualization maps variables to graphical properties such as position, length, colour, size, shape, and transparency. These mappings should reflect the structure of the data and the analytical question.

Matplotlib provides the foundational plotting system used throughout much of the Python scientific ecosystem. Its object-oriented interface distinguishes the complete Figure from individual Axes plotting areas and provides a clear foundation for constructing both simple and multi-panel graphics.

Scatter plots are appropriate for investigating relationships between numerical variables. Line plots emphasize change along an ordered dimension. Bar plots compare numerical quantities across categories, while histograms and box plots describe numerical distributions in different ways.

Graph type has interpretive consequences. Connecting observations with a line suggests ordered progression; bar length implies magnitude relative to a baseline; aggregation can conceal within-group distributions.

Multiple plots and faceting provide alternatives when a single plotting area becomes too complex.

Seaborn builds on Matplotlib and provides higher-level statistical abstractions, particularly for tabular data containing combinations of numerical and categorical variables.

Axis scales determine how numerical differences or ratios are represented visually. Logarithmic scales can be useful when values span several orders of magnitude, but their interpretation differs from that of linear scales.

Time-series plots exploit the intrinsic ordering of temporal observations and can reveal trends, seasonal behaviour, changes in variability, and unusual events.

Pie charts represent part-to-whole relationships using angles and areas, but bar plots often provide more accurate comparisons when differences between categories matter.

A technically valid plot can still be misleading. Important concerns include truncated axes, excessive graphical complexity, overplotting, inappropriate aggregation, dual axes, missing uncertainty, and unjustified causal interpretation.

Network visualizations represent relationships among entities rather than observations positioned directly according to measured numerical axes. The geometry produced by a layout algorithm should therefore be distinguished from the underlying network relationships.

In data science and machine learning, visualization supports data understanding, model diagnosis, performance comparison, and communication of analytical results.

Ultimately, effective visualization depends less on memorizing plotting commands than on understanding the relationship between **data**, **graphical encoding**, **statistical meaning**, and the **comparison the viewer is being asked to make**.

## 25 Problems

The following problems review the main concepts introduced in this chapter. They progress from basic plot construction to graphical interpretation, statistical visualization, model diagnostics, and visualization design.

### 25.1 Problem 1 — Basic Line Plot

Given:

```
1 x = np.array([0, 1, 2, 3, 4])
2 y = np.array([0, 1, 4, 9, 16])
```

create a line plot of x versus y.

The plot should include:

- an appropriate title;
- an x-axis label;
- a y-axis label.

Use Matplotlib's object-oriented interface:

```
1 fig, ax = plt.subplots()
```

Explain which variable is mapped to horizontal position and which is mapped to vertical position.

## 25.2 Problem 2 — Visualizing a Mathematical Function

Create a plot of:

$$f(x) = x^2$$

for:

$$-5 \leq x \leq 5.$$

Generate sufficiently many values of  $x$  using `np.linspace()` so that the curve appears smooth.

Then create a second version using:

- a dashed line;
- markers at the sampled values.

Explain why increasing the number of sampled  $x$  values changes the visual smoothness of the curve even though the mathematical function itself has not changed.

## 25.3 Problem 3 — Comparing Mathematical Functions

Plot the functions:

$$f(x) = x^2$$

and:

$$g(x) = x^3$$

for:

$$-5 \leq x \leq 5.$$

Place both functions on the same Axes.

The plot should include:

- different line styles or markers;
- a legend;
- axis labels;

- an appropriate title.

Then explain one advantage and one disadvantage of placing both functions on the same Axes rather than using separate subplots.

## 25.4 Problem 4 — Figure Configuration

Using the functions from Problem 3:

1. create a figure with size (10, 6);
2. set the horizontal range to:

$$-6 \leq x \leq 6;$$

3. set the vertical range to:

$$-10 \leq y \leq 10;$$

4. add a grid;
5. add an informative title and axis labels.

Explain why restricting the vertical axis changes what parts of the quadratic and cubic functions remain visible.

## 25.5 Problem 5 — Scatter Plot

Suppose:

```

1 height = np.array([
2     174.3, 153.5, 162.1, 157.9, 174.8,
3     169.2, 172.0, 160.4, 152.8, 163.3
4 ])
5
6 weight = np.array([
7     65.7, 59.2, 57.3, 61.5, 77.3,
8     90.7, 85.4, 63.1, 73.2, 105.5
9 ])

```

Create a scatter plot of height versus weight.

Then describe what can legitimately be inferred from the plot regarding:



- direction of association;
- possible unusual observations;
- causality.

Explain why a visible association between height and weight does not by itself establish a causal relationship.

## 25.6 Problem 6 — Adding a Group Encoding

Suppose each observation in Problem 5 belongs to one of two groups:

```
1 group = np.array([
2     "A", "A", "B", "A", "B",
3     "B", "B", "A", "A", "B"
4 ])
```

Create a scatter plot that distinguishes the groups visually.

You may use separate calls to `ax.scatter()` or another appropriate approach.

Add a legend.

Explain which data variable is being encoded using:

- horizontal position;
- vertical position;
- colour.

## 25.7 Problem 7 — Continuous Colour Encoding

Generate:

```
1 rng = np.random.default_rng(42)
2
3 x = rng.normal(size=100)
4 y = rng.normal(size=100)
```

Define:

```
1 distance = np.sqrt(x**2 + y**2)
```

Create a scatter plot where:

- x determines horizontal position;
- y determines vertical position;
- distance determines colour.

Add an appropriate colour bar.

Explain why a colour bar is necessary when colour represents a continuous numerical variable.

## 25.8 Problem 8 — Marker Size as an Encoding

Given:

```
1 x = np.array([1, 2, 3, 4, 5])
2 y = np.array([4, 6, 5, 8, 7])
3 magnitude = np.array([10, 20, 40, 80, 160])
```

create a scatter plot in which magnitude controls marker size.

Then answer:

1. What numerical variable is represented by marker area?
2. Why is marker size generally less precise for quantitative comparison than position?
3. In what type of visualization might size still be useful?

## 25.9 Problem 9 — Overplotting

Generate:

```
1 rng = np.random.default_rng(42)
2
3 x = rng.normal(size=5000)
4 y = 0.7 * x + rng.normal(size=5000)
```

Create two scatter plots:

1. one using the default opacity;
2. one using an appropriate value of alpha.

Compare them.

Explain how transparency helps reveal regions of high observation density.

## 25.10 Problem 10 — Bar Plot

Suppose:

```
1 models = np.array([
2     "Linear",
3     "Random Forest",
4     "Gradient Boosting",
5     "Neural Network"
6 ])
7
8 rmse = np.array([
9     5.4,
10    4.2,
11    3.8,
12    4.0
13 ])
```

Create a horizontal bar plot comparing RMSE across models.

Sort the models from lowest to highest RMSE before plotting.

Then explain:

- what bar length represents;
- why sorting can improve comparison;
- why lower values are preferable in this particular example.

## 25.11 Problem 11 — Truncated Bar Chart

Consider two model accuracies:

```
1 models = ["Model A", "Model B"]
2 accuracy = [0.96, 0.98]
```

Create:

1. a bar chart whose y-axis begins at zero;
2. a bar chart whose y-axis begins at 0.95.

Compare the visual impression created by the two graphs.

Which representation gives a more appropriate impression of relative magnitude when bar length is the encoding?

Explain why.

## 25.12 Problem 12 — Histogram and Bin Selection

Generate:

```
1 rng = np.random.default_rng(42)
2
3 values = rng.normal(
4     loc=0,
5     scale=1,
6     size=1000
7 )
```

Create three histograms using:

```
1 10 bins
2 30 bins
3 100 bins
```

Compare the resulting representations.

Explain how:

- too few bins can hide structure;
- too many bins can emphasize random sample variation.

## 25.13 Problem 13 — Distribution Shape

Generate three samples:

```
1 normal_sample = rng.normal(
2     size=1000
3 )
4
5 right_skewed_sample = rng.exponential(
```

```

6     size=1000
7 )
8
9 bimodal_sample = np.concatenate([
10     rng.normal(-2, 0.5, 500),
11     rng.normal(2, 0.5, 500)
12 ])

```

Create one histogram for each distribution using separate subplots.

Describe each distribution in terms of:

- symmetry;
- skewness;
- modality;
- spread.

Explain why the mean and standard deviation alone would not reveal all these differences.

## 25.14 Problem 14 — Frequency Versus Density

Using a normally distributed sample, create:

1. a histogram showing counts;
2. a histogram using density=True.

Explain the difference between the vertical axes.

Why should the height of a density histogram not generally be interpreted directly as the probability of observing one exact value?

## 25.15 Problem 15 — Box Plot

Generate:

```

1 values = rng.normal(
2     loc=50,
3     scale=10,
4     size=200
5 )

```

Add several unusually large observations:

```
1 values = np.concatenate([
2     values,
3     [90, 95, 100]
4 ])
```

Create a box plot.

Then calculate:

```
1 Q1
2 median
3 Q3
4 IQR
```

and the conventional boundaries:

$$Q_1 - 1.5 \text{ IQR}$$

and:

$$Q_3 + 1.5 \text{ IQR}.$$

Explain why observations outside these boundaries should be described as potential outliers rather than automatically classified as errors.

## 25.16 Problem 16 — Comparing Distributions Across Groups

Generate three groups with different means and standard deviations and display them using a single box plot.

For example:

```
1 group_a = rng.normal(50, 5, 200)
2 group_b = rng.normal(60, 10, 200)
3 group_c = rng.normal(55, 3, 200)
```

Compare the groups in terms of:

- median;
- interquartile range;

- overall spread;
- potential extreme observations.

Then explain what information about the distributions is not visible from a box plot.

### 25.17 Problem 17 — Bar Plot or Box Plot?

Suppose you are comparing customer spending across three regions.

You have access to every individual transaction.

Explain what information would be represented by:

1. a bar plot of mean spending by region;
2. a box plot of spending by region;
3. three histograms of spending.

Which representation would you choose if the analytical question were:

Do the complete spending distributions differ substantially between regions?

Justify your answer.

### 25.18 Problem 18 — Multiple Subplots

Given:

```
1 x = np.arange(1, 11)
2 y = x ** 2
```

create a  $2 \times 3$  figure containing:

1. a line plot;
2. a scatter plot;
3. a bar plot;
4. a log-log plot;
5. a semilog-x plot;
6. a semilog-y plot.

Title and label each Axes appropriately.

Use:

```
1 fig, axes = plt.subplots(2, 3)
```

rather than the state-based `plt.subplot()` interface.

Explain what changes when a logarithmic scale is introduced.

### 25.19 Problem 19 — Linear and Logarithmic Scales

Given:

```
1 x = np.array([
2     1,
3     10,
4     100,
5     1000,
6     10000
7 ])
8
9 y = np.array([
10    2,
11    4,
12    8,
13    16,
14    32
15 ])
```

display the data:

1. using a linear x-axis;
2. using a logarithmic x-axis.

Compare the representations.

Explain the distinction between equal **differences** on a linear scale and equal **ratios** on a logarithmic scale.

### 25.20 Problem 20 — Time-Series Plot

Create a sequence of 30 daily dates beginning on January 1, 2026.

Generate a numerical series containing:



- an upward trend;
- random noise.

Plot the series using actual dates on the horizontal axis.

The plot should include:

- a title;
- axis labels;
- readable date labels.

Explain why connecting consecutive observations with lines is meaningful in this context.

### 25.21 Problem 21 — Trend and Seasonality

Construct a synthetic time series:

$$y_t = 0.05t + 2 \sin\left(\frac{2\pi t}{12}\right) + \varepsilon_t,$$

where  $\varepsilon_t$  is random noise.

Generate at least 120 observations and plot the series.

Identify visually:

- the trend;
- the seasonal component;
- the random variation.

Explain why a simple overall mean would hide much of this structure.

### 25.22 Problem 22 — Multiple Time Series

Generate two time series over the same 50 time points:

- one with a stronger positive trend;
- one with a weaker positive trend.

Display them on the same Axes and add a legend.

Then create the same series in two vertically stacked subplots sharing the x-axis.

Compare the two representations.

When might separate panels be clearer than overlaying several series?

### 25.23 Problem 23 — Seaborn Scatter Plot

Create the following DataFrame:

```
1 data = pd.DataFrame({  
2     "feature": [1, 2, 3, 4, 5, 6, 7, 8],  
3     "target": [2.1, 3.0, 4.2, 5.1, 6.5, 7.2, 8.1, 9.0],  
4     "group": ["A", "A", "A", "A", "B", "B", "B", "B"]  
5 })
```

Use Seaborn to create a scatter plot where:

```
1 x="feature"  
2 y="target"  
3 hue="group"
```

Explain how this interface differs conceptually from manually creating separate NumPy arrays for each group.

### 25.24 Problem 24 — Regression Visualization

Using an appropriate synthetic dataset, create a Seaborn regression plot.

The visualization should contain:

- observations;
- a fitted linear relationship;
- its default uncertainty representation.

Then explain why the plot alone does not demonstrate that a linear regression model is appropriate.

Identify at least three additional model properties or diagnostics that should be considered.

### 25.25 Problem 25 — Histogram and Density Estimate

Generate 1000 observations from a normal distribution and create a Seaborn histogram with:

```
1 stat="density"  
2 kde=True
```

Explain the difference between:

- the histogram;
- the kernel density estimate.

Why should the density estimate be understood as a smoothed representation rather than as the raw empirical distribution?

### 25.26 Problem 26 — Faceting

Create a DataFrame containing numerical variables  $x$  and  $y$  and a categorical variable with three groups.

Use Seaborn to create one scatter plot per group using faceting.

Then create a second visualization in which all groups appear on the same Axes using colour.

Compare the two approaches.

Discuss:

- overlap;
- ease of within-group interpretation;
- ease of between-group comparison;
- legend complexity.

### 25.27 Problem 27 — Choosing the Appropriate Plot

For each analytical question below, choose an appropriate visualization and justify your choice.

1. What is the distribution of household income?
2. How does model error vary as training proceeds across epochs?
3. How are height and weight related?
4. How does the distribution of salary differ across departments?
5. Which of five models has the lowest RMSE?
6. How do monthly sales evolve over three years?
7. Does a numerical feature contain extreme observations?
8. How do two continuous predictors relate while observations belong to three categories?

There may be more than one defensible choice. The objective is to justify the graphical representation based on the analytical question.

## 25.28 Problem 28 — Pie Chart Versus Bar Chart

Suppose:

```
1 categories = [  
2     "A",  
3     "B",  
4     "C",  
5     "D"  
6 ]  
7  
8 percentages = [  
9     28,  
10    26,  
11    24,  
12    22  
13 ]
```

Create:

1. a pie chart;
2. a bar chart.

Compare how easily the four categories can be ranked in each representation.

Explain why differences between similar proportions may be easier to perceive using position or bar length than angles and areas.

## 25.29 Problem 29 — Misleading Visualization

Suppose a report shows annual revenue:

```
1 year = [2023, 2024, 2025, 2026]  
2 revenue = [100, 102, 103, 105]
```

Construct two line plots:

1. one with a vertical axis beginning at zero;
2. one restricted approximately to the observed range.

Discuss whether either representation is automatically misleading.

Explain why the interpretation of a restricted axis differs between a line plot and a bar plot.

### 25.30 Problem 30 — Aggregation Can Hide Structure

Generate two groups having approximately the same mean but substantially different distributions.

For example:

- Group A: approximately normal with small variance;
- Group B: bimodal with larger variance.

Create:

1. a bar plot of the group means;
2. box plots of the groups;
3. histograms or density plots of the groups.

Compare the conclusions suggested by the three graphical representations.

What information disappeared when the data were reduced to a single mean per group?

### 25.31 Problem 31 — Uncertainty

Suppose three models have estimated accuracies:

```
1 accuracy = np.array([
2     0.82,
3     0.84,
4     0.86
5 ])
6
7 standard_error = np.array([
8     0.01,
9     0.03,
10    0.02
11 ])
```

Construct a graphical comparison that includes the uncertainty associated with each estimate.

Explain why comparing only the point estimates may communicate more certainty than the analysis supports.

### 25.32 Problem 32 — Three-Dimensional Parametric Plot

Define:

$$x(t) = \sin(t),$$

$$y(t) = \cos(t),$$

for:

$$0 \leq t \leq 10\pi.$$

Create a three-dimensional plot of:

$$(x(t), y(t), t).$$

Include:

- labels for all three axes;
- an appropriate title.

Then describe geometrically the structure traced by the curve.

### 25.33 Problem 33 — Three-Dimensional Scatter Plot

Generate 50 random values for each of:

```
1 x
2 y
3 z
```

using a reproducible random number generator.

Create a three-dimensional scatter plot.

Then explain one limitation of interpreting 3D scatter plots on a static two-dimensional page.

Why can occlusion and viewing angle make numerical relationships harder to judge?

### 25.34 Problem 34 — Creating a Meshgrid

Given:

```
1 x = np.array([1, 2, 3, 4])
2 y = np.array([3, 4, 5])
```

use:

```
1 np.meshgrid()
```

to construct:

```
1 X, Y
```

Determine:

```
1 X.shape
2 Y.shape
```

and explain what each pair:

```
1 X[i, j], Y[i, j]
```

represents.

### 25.35 Problem 35 — Surface Plot

Create a grid covering:

$$-5 \leq x \leq 5$$

and:

$$-5 \leq y \leq 5.$$

Evaluate:

$$f(x, y) = \sin(x) \cos(y).$$

Create a three-dimensional surface plot.

Use a sufficiently fine discretization so that the surface appears smooth.

Explain the roles of `meshgrid()`, `X`, `Y`, and `Z` in constructing the surface.

### 25.36 Problem 36 — Surface and Wireframe Comparison

Using the data from Problem 35, create a figure containing:

1. a wireframe representation;
2. a surface representation.

Use a  $1 \times 2$  arrangement.

Compare the information emphasized by each representation.

Which makes the shape of the surface easier to perceive?

Which more clearly exposes the underlying grid structure?

### 25.37 Problem 37 — Network Construction

Create an undirected NetworkX graph containing the nodes:

```
1 A, B, C, D, E
```

and edges:

```
1 A-B
2 A-C
3 B-D
4 C-D
5 D-E
```

Then:

1. list all nodes;
2. list all edges;
3. determine the neighbours of D;
4. calculate the degree of every node.



Identify the node with the highest degree.

### 25.38 Problem 38 — Network Layout

Using the graph from Problem 37, create two visualizations:

1. using the default layout;
2. using `nx.spring_layout()` with a fixed seed.

Compare the node positions.

Explain why the geometric distance between two nodes in a spring-layout network should not automatically be interpreted as a directly measured numerical distance.

### 25.39 Problem 39 — Weighted Network

Create a weighted graph using:

```
1 weighted_edges = [  
2     ("A", "B", 2.0),  
3     ("A", "C", 1.5),  
4     ("B", "D", 3.0),  
5     ("C", "D", 2.5),  
6     ("D", "E", 4.0)  
7 ]
```

Inspect the edge weights.

Then explain at least three different substantive meanings these weights might have depending on the application.

Why can the same network structure have different interpretations depending on what the weights represent?

### 25.40 Problem 40 — Residual Plot

Suppose:

```

1 targets = np.array([
2     2.0, 3.0, 4.2, 5.1,
3     6.4, 7.0, 8.5, 9.0
4 ])
5
6 predictions = np.array([
7     2.1, 2.9, 4.0, 5.4,
8     6.1, 7.2, 8.2, 9.1
9 ])

```

Calculate:

```

1 residuals = targets - predictions

```

and create a scatter plot of residuals versus predictions.

Add a horizontal reference line at:

$$e = 0.$$

Then explain what types of visual patterns might suggest:

- systematic model bias;
- nonlinearity;
- changing residual variance.

## 25.41 Problem 41 — Detecting Nonlinearity from Residuals

Generate:

```

1 x = np.linspace(-3, 3, 100)
2
3 targets = x**2 + rng.normal(
4     scale=0.5,
5     size=100
6 )
7
8 predictions = (
9     np.mean(targets)

```

```
10 + np.zeros_like(x)
11 )
```

Construct an appropriate residual plot.

Describe the structure you observe.

Why would a strong systematic pattern in residuals suggest that the fitted model has failed to capture part of the signal?

### 25.42 Problem 42 — Model Comparison

Suppose:

```
1 models = np.array([
2     "Linear Regression",
3     "Random Forest",
4     "Gradient Boosting",
5     "Neural Network"
6 ])
7
8 rmse = np.array([
9     6.1,
10    4.5,
11    3.9,
12    4.3
13 ])
```

Create an appropriate visualization of model performance.

Then answer:

1. Which model appears best according to RMSE?
2. Does the visualization establish that it is the best model overall?
3. What other considerations might affect model selection?

### 25.43 Problem 43 — Learning Curves

Suppose:

```

1 epochs = np.arange(1, 31)
2
3 training_loss = (
4     np.exp(-0.15 * epochs)
5     + 0.05
6 )
7
8 validation_loss = (
9     np.exp(-0.12 * epochs)
10    + 0.12
11    + 0.002 * epochs
12 )

```

Plot both curves.

Add appropriate:

- labels;
- title;
- legend.

Describe the behaviour of training and validation loss.

At what stage might the relationship between the curves begin to suggest overfitting?

## 25.44 Problem 44 — Visualizing Class Imbalance

Suppose a binary classification dataset contains:

```

1 class_0 = 9500
2 class_1 = 500

```

Choose and construct an appropriate visualization of class frequency.

Calculate the proportions represented by each class.

Explain why identifying this imbalance may be important before training or evaluating a classifier.

### 25.45 Problem 45 — Visual Encoding Critique

Suppose a scatter plot attempts to represent six variables simultaneously:

- x-position;
- y-position;
- colour;
- size;
- marker shape;
- transparency.

Critique this design.

Explain why adding more information to a plot does not necessarily make the plot more informative.

Propose an alternative design using:

- fewer encodings;
- multiple panels;
- or faceting.

### 25.46 Problem 46 — Dual-Axis Critique

A report plots monthly advertising expenditure on the left y-axis and monthly revenue on the right y-axis.

Explain why changing the ranges of the two axes can visually strengthen or weaken the apparent relationship.

Propose at least two alternative graphical representations that reduce this interpretive risk.

### 25.47 Problem 47 — Correlation and Causality

Suppose a scatter plot shows a strong positive relationship between two variables.

Provide at least four possible explanations for the observed relationship, including causal and non-causal possibilities.

Explain why the plot alone cannot identify which explanation is correct.

### 25.48 Problem 48 — Design an Exploratory Visualization Strategy

Suppose you receive a dataset containing:

- 10,000 observations;
- five numerical features;
- two categorical features;
- a binary target;
- a timestamp.

Before fitting a machine learning model, propose a sequence of visualizations that would help you investigate:

- numerical distributions;
- outliers;
- relationships between numerical features;
- categorical frequencies;
- class imbalance;
- relationships between features and the target;
- temporal structure.

For each proposed visualization, state the analytical question it is intended to answer.

### 25.49 Problem 49 — Diagnose a Poor Visualization

Consider the following description:

A scatter plot contains 100,000 observations. Every point is fully opaque and relatively large. Five categories are represented by similar shades of the same colour. Marker size represents income, marker shape represents region, and another variable is represented using annotations on individual observations. A large legend occupies one-third of the plotting area.

Identify the major visualization problems.

Propose a redesign that preserves the important analytical information while reducing visual complexity.

## 25.50 Problem 50 — Integrated Visualization Reasoning

Suppose you are evaluating a regression model and have:

```
1 X
2 y
3 predictions
4 residuals
```

where  $X$  contains several numerical features.

Design a small diagnostic visualization set containing no more than four plots.

Your plots should help investigate:

- prediction quality;
- residual behaviour;
- target distribution;
- at least one important relationship between a feature and the target.

For each plot:

1. identify the graphical form;
2. identify the variables being encoded;
3. explain what analytical question it answers;
4. describe at least one pattern that would deserve further investigation.

The objective is not to maximize the number of plots but to construct a small, coherent diagnostic set.